

POC-01: Secure Batch Ingestion with Azure Data Factory, ADLS Gen2 and Azure SQL

Beginner Implementation Guide

Manual Azure setup, Python upload, troubleshooting, verification and Terraform recreation

How to use this guide

This handbook rebuilds the completed POC from scratch in the same order a beginner should follow. The manual Azure implementation is the baseline. Python is used to land test data securely. Terraform is presented afterward as a separate infrastructure-as-code recreation path.

Important: two implementation variants exist

The completed manual pipeline uses a 5-row CSV with an `order_date` column and a simple MERGE/log design. The repository also contains an advanced 30-row lab using `order_ts`, rejects, file-level control and watermark logic. Do not mix the two schemas accidentally.

Recommended learning sequence

- Build the Azure resources manually and understand what each service does.
- Create and test ADF linked services, datasets and the pipeline.
- Run the 5-row manual CSV end to end and verify SQL, archive and monitoring.
- Run the Python uploader using Microsoft Entra authentication.
- Perform idempotency, missing-file and RBAC negative tests.
- Recreate the infrastructure with Terraform using separate names or after cleanup.
- Optionally migrate to the advanced 30-row validation/watermark variant.

Contents - Part 1

- 1. Objective, architecture and Azure concepts
- 2. Prerequisites, naming and cost guardrails
- 3. Manual Azure setup: Resource Group and ADLS Gen2
- 4. Manual Azure setup: Azure SQL, firewall and Microsoft Entra
- 5. Manual Azure setup: Data Factory, Managed Identity and RBAC
- 6. SQL tables and idempotent MERGE
- 7. ADF linked services and parameterized datasets
- 8. ADF pipeline activities and expressions

Contents - Part 2

- 9. Python Azure authentication and ADLS upload
- 10. End-to-end testing and verification
- 11. Errors encountered and exact fixes
- 12. Repository files: what to run and what not to mix
- 13. Terraform recreation: complete workflow and source
- 14. Monitoring, security, GitHub hygiene and cleanup
- 15. Interview questions and completion checklist
- 16. Screenshot evidence integrated through the guide

1. Objective and business scenario

A retail company receives daily order CSV files. The platform must land raw input safely, orchestrate processing, load relational staging, merge business rows without duplicates, preserve processed source files for audit and expose execution evidence.

Business requirements

- Accept a new file in ADLS landing.
- Check that the expected file exists before processing.
- Load into SQL staging, then merge on the business key.
- Re-running the same input must not duplicate curated orders.
- Archive the raw file only after successful processing.
- Log execution status and verify activity results independently.

Target data flow

Architecture flow



Azure concepts used in this POC

Resource Group	Lifecycle boundary for the POC. Deleting it is the simplest cost-control cleanup.
ADLS Gen2	StorageV2 account with hierarchical namespace. Holds landing, archive and quarantine zones.
Azure Data Factory	Orchestrates file checks, copy, SQL procedure execution, archive and delete operations.
Azure SQL Database	Contains staging, curated orders and ETL logging/control objects.
Managed Identity	Lets ADF request Microsoft Entra tokens without storing an application password.
Azure RBAC	Grants the ADF identity and developer identity only the required ADLS data access.
Azure Monitor / ADF Monitor	Provides run status, activity duration, row counts and failure details.
Terraform	Recreates the Azure foundation as code after the manual learning path is understood.

2. Prerequisites, names and cost guardrails

Windows prerequisites

- Python 3.12 is fine for this POC.
- Poetry environment with azure-identity and azure-storage-file-datalake.
- Azure CLI is a separate Windows application and is required for the az command.
- Git and VS Code are recommended.
- Terraform 1.6+ is required only for the IaC phase.

Neutral naming pattern

Resource Group	rg-azde-poc01-dev
Storage Account	stazdepoc01
Data Factory	adf-azde-poc01-
SQL logical server	sql-azde-poc01-
SQL Database	sqlldb-azde-poc01-dev
Key Vault	kv-azde-poc01-

Cost rules

- Use a tiny development Azure SQL configuration available in your region.
- Run ADF only on demand.
- Keep test files tiny.
- Treat Log Analytics and Private Endpoints as optional learning additions.
- Delete the POC Resource Group after evidence capture.

3. Manual setup - Create the Resource Group

- Sign in to Azure Portal and confirm the correct tenant and subscription.
- Open Resource groups and choose Create.
- Create rg-azde-poc01-dev in a region supported by all required services.
- Add project, environment, owner and autoDelete tags.
- Optionally create a small Cost Management budget with email thresholds.

Beginner tip

A Resource Group is not a folder only. It is also a lifecycle and access-management boundary. Keeping this POC isolated makes cleanup safer.

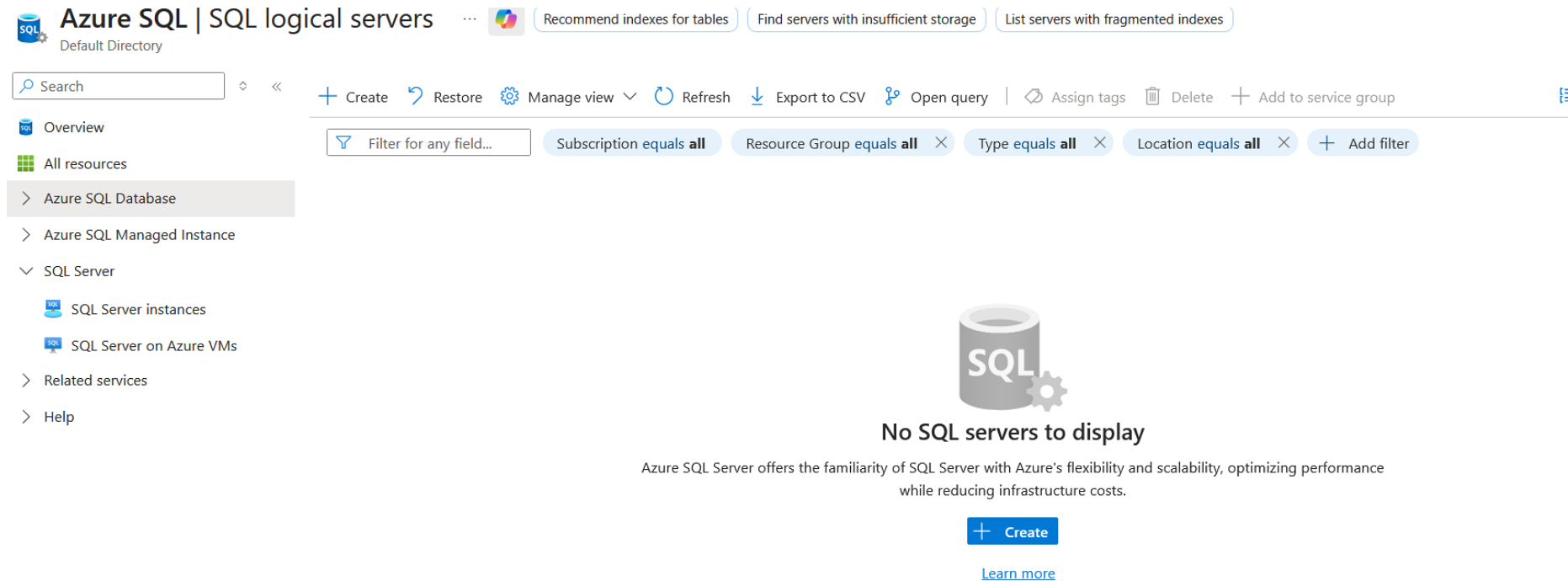
Manual setup - Create ADLS Gen2

- Create a StorageV2 account in the POC Resource Group.
- Use Standard performance and LRS for the small lab.
- Enable Hierarchical namespace.
- Keep anonymous/public blob access disabled.
- Create private containers named landing, archive and quarantine.
- Use landing/orders/2026/08/28/orders_001.csv as the manual sample path.

Why three containers?

landing is the active ingestion zone; archive preserves successfully processed raw files; quarantine holds invalid or failed evidence.

Manual Azure setup - Azure SQL service



Open the Azure SQL service and begin creation of the logical server/database.
What this proves: This is the starting point for the relational side of the POC.

Issue - SQL region/capacity restriction

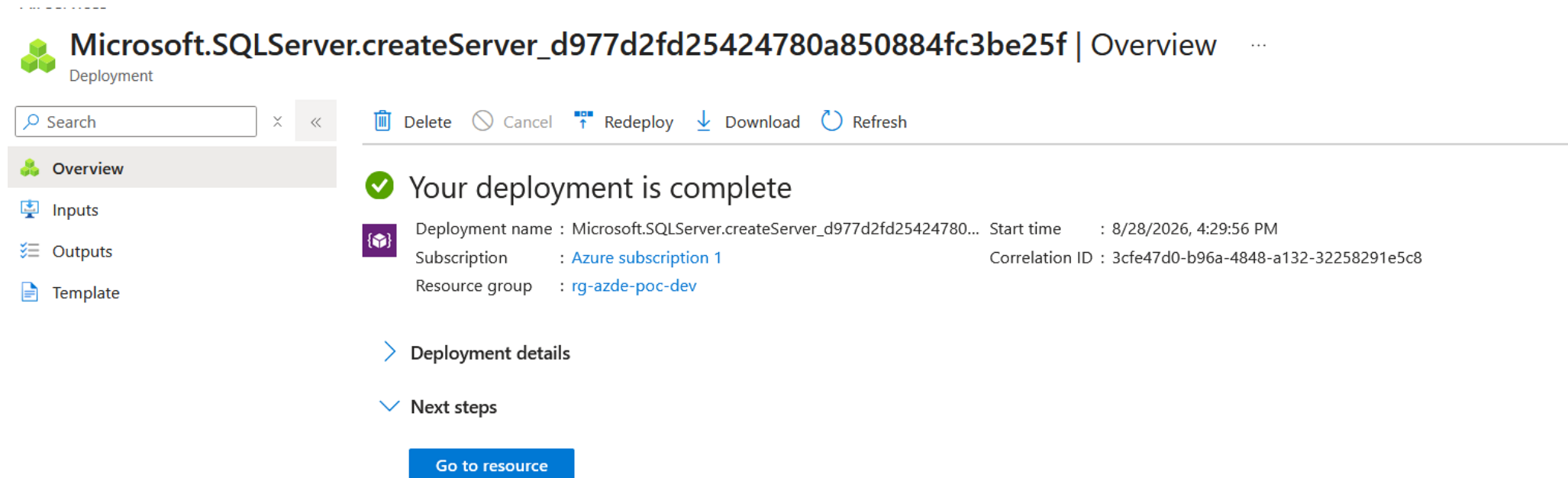
The screenshot shows the 'Create SQL Database Server' page in the Microsoft Azure portal. The page is titled 'Create SQL Database Server' and includes a warning message: 'Changing Basic options may reset selections you have made. Review all options prior to creating the resource.' Below this, the 'Subscription' is set to 'Azure subscription 1' and the 'Resource group' is 'rg-azde-poc-dev'. Under 'Server details', the 'Server name' is 'sql-azde-poc-irshad-01' and the 'Location' is '(US) East US'. A red error message is displayed below the location dropdown: 'Your subscription does not have access to create a server in the selected region. For the latest information about region availability for your subscription, go to aka.ms/sqlcapacity. Please try another region or create a support ticket to request access.' At the bottom, there are buttons for 'Review + create' and 'Next: Networking >'. A blue banner at the bottom of the form area states: 'Azure Active Directory (Azure AD) is now Microsoft Entra ID. Learn more'.

A region/capacity restriction was encountered while creating Azure SQL. What this proves: Azure SQL creation may be limited by subscription or regional capacity.

Issue and fix

Choose another region supported by the subscription and the selected SQL configuration, then create the server/database there. Keep related resources in the same region where practical.

Manual Azure setup - SQL logical server created



The screenshot displays the Azure portal interface for a deployment. At the top, the title bar shows the deployment name 'Microsoft.SQLServer.createServer_d977d2fd25424780a850884fc3be25f' and the 'Overview' tab. Below the title bar, there is a search bar and a toolbar with icons for Delete, Cancel, Redeploy, Download, and Refresh. The left sidebar contains a navigation menu with 'Overview' (selected), 'Inputs', 'Outputs', and 'Template'. The main content area features a green checkmark icon and the text 'Your deployment is complete'. Below this, the deployment details are listed: 'Deployment name : Microsoft.SQLServer.createServer_d977d2fd25424780...', 'Start time : 8/28/2026, 4:29:56 PM', 'Subscription : Azure subscription 1', and 'Resource group : rg-azde-poc-dev'. The 'Correlation ID' is also provided as '3cfe47d0-b96a-4848-a132-32258291e5c8'. At the bottom of the main content area, there are two expandable sections: 'Deployment details' and 'Next steps', and a blue button labeled 'Go to resource'.

Microsoft.SQLServer.createServer_d977d2fd25424780a850884fc3be25f | Overview

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : Microsoft.SQLServer.createServer_d977d2fd25424780... Start time : 8/28/2026, 4:29:56 PM

Subscription : Azure subscription 1 Correlation ID : 3cfe47d0-b96a-4848-a132-32258291e5c8

Resource group : rg-azde-poc-dev

> Deployment details

✓ Next steps

Go to resource

The Azure SQL logical server deployment completed successfully.
What this proves: The server resource now exists and can host the POC database.

Manual Azure setup - Configure SQL Database

Manage all your resources.

Subscription ⓘ Azure subscription 1

Resource group ⓘ rg-azde-poc-dev

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources.

Database name * Enter database name

Server ⓘ sql-azde-poc-irshad-01 (Central India)

Want to use SQL elastic pool? ⓘ ☐ Yes ☒ No

Workload environment ☐ Development ☒ Production

ESTIMATED COMPUTE COST / MONTH 586.70 USD
STORAGE COST / GB / MONTH 0.28 USD

STORAGE NOTES
¹ In the Hyperscale tier, storage costs are calculated based on actual allocation. Allocated space increases automatically as needed, up to 100 TB.

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources.

Database name * Enter database name

Server ⓘ sql-azde-poc-irshad-01 (Central India)

Want to use SQL elastic pool? ⓘ ☐ Yes ☒ No

Workload environment ☐ Development ☒ Production

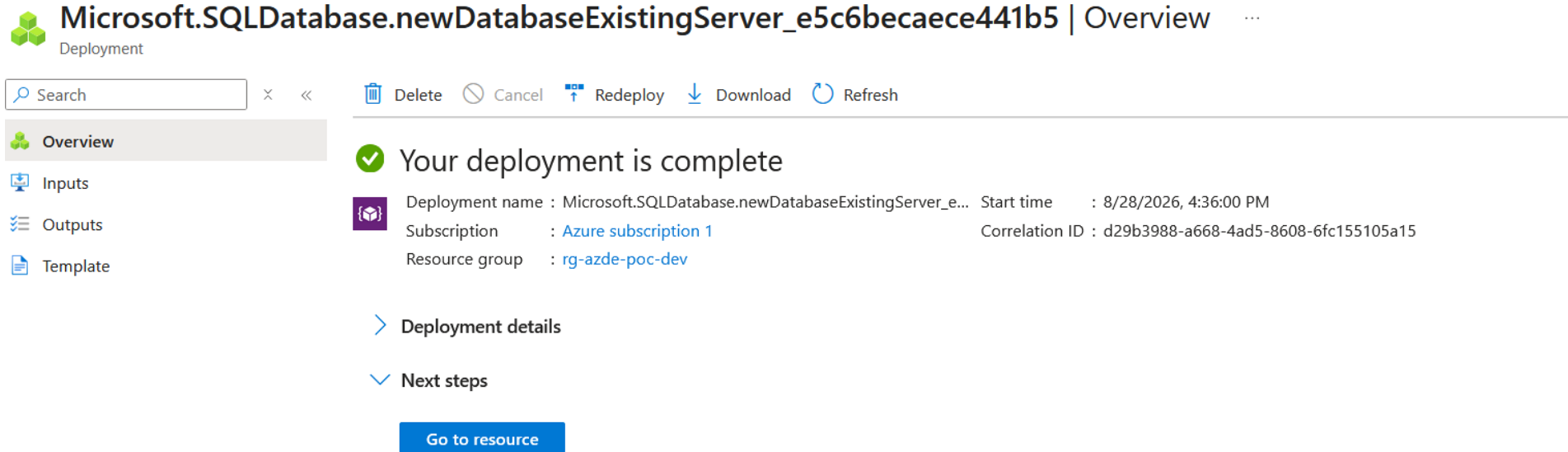
Default settings provided for Production workloads. Configurations can be modified as needed.

Review + create **Next : Networking >**

Configure the SQL Database under the selected logical server.

What this proves: Use the small lab database name and a development SKU available in the chosen region.

Manual Azure setup - SQL Database created



The screenshot displays the Azure Portal interface for a deployment. At the top, the title bar reads "Microsoft.SQLDatabase.newDatabaseExistingServer_e5c6becaece441b5 | Overview". Below this, a search bar and a toolbar with icons for Delete, Cancel, Redeploy, Download, and Refresh are visible. The left sidebar contains a navigation menu with "Overview" (selected), "Inputs", "Outputs", and "Template". The main content area features a green checkmark icon and the text "Your deployment is complete". Below this, deployment details are listed: "Deployment name : Microsoft.SQLDatabase.newDatabaseExistingServer_e...", "Subscription : Azure subscription 1", "Resource group : rg-azde-poc-dev", "Start time : 8/28/2026, 4:36:00 PM", and "Correlation ID : d29b3988-a668-4ad5-8608-6fc155105a15". At the bottom, there are expandable sections for "Deployment details" and "Next steps", and a blue button labeled "Go to resource".

Microsoft.SQLDatabase.newDatabaseExistingServer_e5c6becaece441b5 | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : Microsoft.SQLDatabase.newDatabaseExistingServer_e... Start time : 8/28/2026, 4:36:00 PM

Subscription : [Azure subscription 1](#) Correlation ID : d29b3988-a668-4ad5-8608-6fc155105a15

Resource group : [rg-azde-poc-dev](#)

> Deployment details

✓ Next steps

Go to resource

The POC Azure SQL Database deployment completed successfully.

What this proves: The target database is ready for firewall, Microsoft Entra and schema configuration.

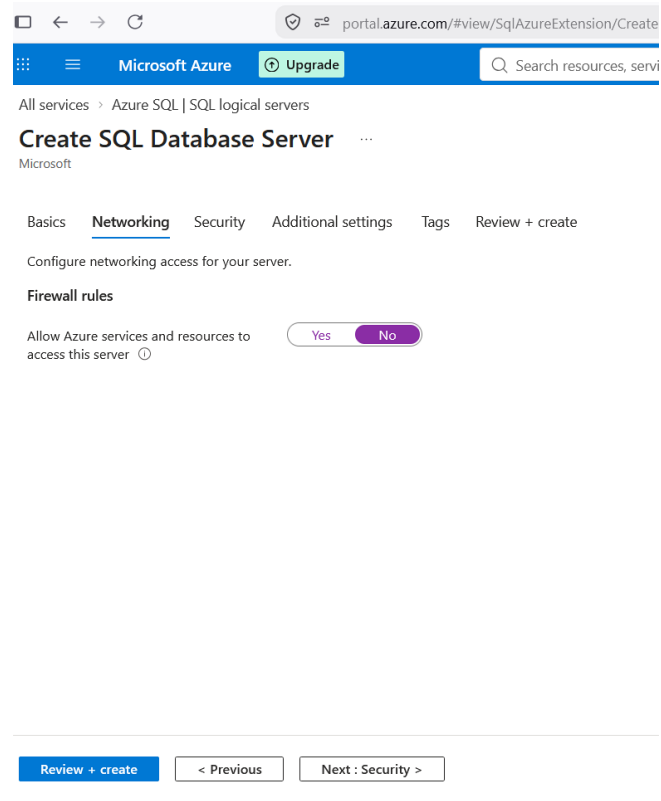
4. Manual setup - SQL networking and firewall

- Open the Azure SQL logical server, not only the database.
- Open Networking.
- For this beginner public-endpoint lab, enable Allow Azure services and resources to access this server.
- Add your current client IP when browser/query/client access from your laptop requires it.
- Save the networking changes.

Production note

This public-endpoint exception is a beginner-lab simplification. A hardened design uses private connectivity and still requires identity/database authorization.

SQL networking evidence



Azure SQL server Networking configuration showing the beginner-lab Azure services firewall option. What this proves: This setting was required for the ADF Azure Integration Runtime to reach the SQL server in the manual lab.

5. Manual setup - Create Azure Data Factory

- Create a Data Factory in the POC Resource Group.
- Configure Git later for the first manual run.
- Open the deployed Data Factory and launch ADF Studio.
- Use the system-assigned Managed Identity for ADLS and SQL connectivity.

Create Azure Data Factory

Create Data Factory ...

Basics Git configuration Networking Advanced Tags Review + create

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ

Azure subscription 1

Resource group * ⓘ

Create new

Instance details

Name * ⓘ

Region * ⓘ

(US) East US

Version * ⓘ

V2

i We recommend you **upgrade to Fabric Data Factory** for a modern, unified data integration experience. [Try Fabric instead!](#)

Previous Next Review + create

Data Factory Basics configuration during manual creation.
What this proves: The factory is placed in the same POC lifecycle boundary.

Azure Data Factory deployment completed

 **CreateDataFactory-20260828151044** | Overview ...

Deployment

×

«

Delete

Cancel

Redeploy

Download

Refresh

Overview

Inputs

Outputs

Template

Your deployment is complete

Deployment name : CreateDataFactory-20260828151044

Subscription : [Azure subscription 1](#)

Resource group : [rg-azde-poc-dev](#)

Start time : 8/28/2026, 3:16:35 PM

Correlation ID : d3e345d7-e13a-47fc-9fd3-c0b92c20682c

> Deployment details

✓ Next steps

Go to resource

Cos...

Get...

pre...

[Set...](#)

Mic...

Sec...

[Go...](#)

Fre...

[Sta...](#)

Wo...

The Data Factory resource deployed successfully.

What this proves: The next steps are RBAC, SQL database principal creation and ADF authoring.

Grant Storage data-plane permission

stazdepocirshad01 | Access Control (IAM)

Storage account

Search

+ Add

Download role assignments

Edit columns

Refresh

Delete

Feedback

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

Data storage

Security + networking

Data management

Settings

Monitoring

Automation

Help

Check access

Role assignments

Roles

Deny assignments

Number of role assignments for this subscription

2

5000

Search by name or email

Type : All

Role : All

Scope : All scopes

Group by : Role

All (2)

Job function roles (1)

Privileged administrator roles (1)

☐	Name ↑↓	Type ↑↓	Role ↑↓	Scope ↑↓	Condition ↑↓
▼ Owner (1)					
☐	 irshad alam 0cabca04-45aa-440e-9ce9-4b0...	User	Owner	 Subscription (Inherited)	None
▼ Storage Blob Data Contributor (1)					
☐	 adf-azde-poc-irshad-01 2c4991cc-10bc-4412-8d7c-7d...	Managed identity	Storage Blob Data Contributor	 This resource	Add

Showing 1 - 2 of 2 results.

Storage Account IAM role assignments for the developer and/or managed identity.

What this proves: The Python uploader and ADF are separate identities and need data-plane permission independently.

Verify Storage Blob Data Contributor

stazdepocirshad01 | Access Control (IAM)

Storage account

Search

+ Add

Download role assignments

Edit columns

Refresh

Delete

Feedback

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

Data storage

Security + networking

Data management

Settings

Monitoring

Automation

Help

Check access

Role assignments

Roles

Deny assignments

Number of role assignments for this subscription

2

5000

Search by name or email

Type : All

Role : All

Scope : All scopes

Group by : Role

All (2)

Job function roles (1)

Privileged administrator roles (1)

☐	Name ↑↓	Type ↑↓	Role ↑↓	Scope ↑↓	Condition ↑↓
Owner (1)					
☐	irshad alam 0cabca04-45aa-440e-9ce9-4b0... User		Owner	Subscription (Inherited)	None
Storage Blob Data Contributor (1)					
☐	adf-azde-poc-irshad-01 2c4991cc-10bc-4412-8d7c-7d... Managed identity		Storage Blob Data Contributor	This resource	Add

Showing 1 - 2 of 2 results.

Add or remove favorites by pressing Ctrl+L+Shift+F

Storage IAM evidence for Storage Blob Data Contributor.

What this proves: ADF needs read, write and delete capability for landing/archive behavior.

Create the manual SQL objects

The completed manual pipeline uses an order_date schema. Run these objects in the POC database before authoring the ADF copy and stored procedure activities.

Manual DDL - logging and staging

```
IF OBJECT_ID('dbo.etl_file_log', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.etl_file_log (
        log_id INT IDENTITY(1,1) PRIMARY KEY,
        file_name VARCHAR(255) NOT NULL,
        file_size_bytes BIGINT,
        status VARCHAR(50) NOT NULL,
        rows_inserted INT DEFAULT 0,
        rows_updated INT DEFAULT 0,
        started_at DATETIME2,
        completed_at DATETIME2,
        error_message NVARCHAR(MAX)
    );
END;

IF OBJECT_ID('dbo.orders_stg', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.orders_stg (
```


Manual DDL - logging and staging - Continued

```
    order_id INT, customer_id INT, order_date VARCHAR(50),  
    product_id INT, quantity INT, unit_price DECIMAL(10,2),  
    status VARCHAR(50)  
);  
END;
```

Create the manual curated table

dbo.orders DDL

```
IF OBJECT_ID('dbo.orders', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.orders (
        order_id INT PRIMARY KEY,
        customer_id INT,
        order_date DATE,
        product_id INT,
        quantity INT,
        unit_price DECIMAL(10,2),
        total_amount AS (quantity * unit_price) PERSISTED,
        status VARCHAR(50),
        source_file VARCHAR(255),
        created_at DATETIME2 DEFAULT SYSUTCDATETIME(),
        updated_at DATETIME2 DEFAULT SYSUTCDATETIME()
    );
END;
```

Create the manual MERGE stored procedure

```
CREATE OR ALTER PROCEDURE dbo.usp_merge_orders
    @p_file_name VARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;

    MERGE dbo.orders AS tgt
    USING (
        SELECT order_id, customer_id, TRY_CONVERT(DATE, order_date) AS order_date,
               product_id, quantity, unit_price, status
        FROM dbo.orders_stg
    ) AS src
    ON tgt.order_id = src.order_id
    WHEN MATCHED THEN UPDATE SET
        tgt.customer_id = src.customer_id, tgt.order_date = src.order_date,
        tgt.product_id = src.product_id, tgt.quantity = src.quantity,
        tgt.unit_price = src.unit_price, tgt.status = src.status,
        tgt.source_file = @p_file_name, tgt.updated_at = SYSUTCDATETIME()
    WHEN NOT MATCHED THEN INSERT
        (order_id, customer_id, order_date, product_id, quantity, unit_price,
```

Create the manual MERGE stored procedure - Continued

```
        status, source_file, created_at, updated_at)
VALUES
    (src.order_id, src.customer_id, src.order_date, src.product_id, src.quantity,
     src.unit_price, src.status, @p_file_name, SYSUTCDATETIME(), SYSUTCDATETIME());
END;
```

Create the ADF Managed Identity database user

Set a Microsoft Entra administrator on the Azure SQL logical server. Connect to the POC database as that administrator, then create a contained user whose name exactly matches the Data Factory resource.

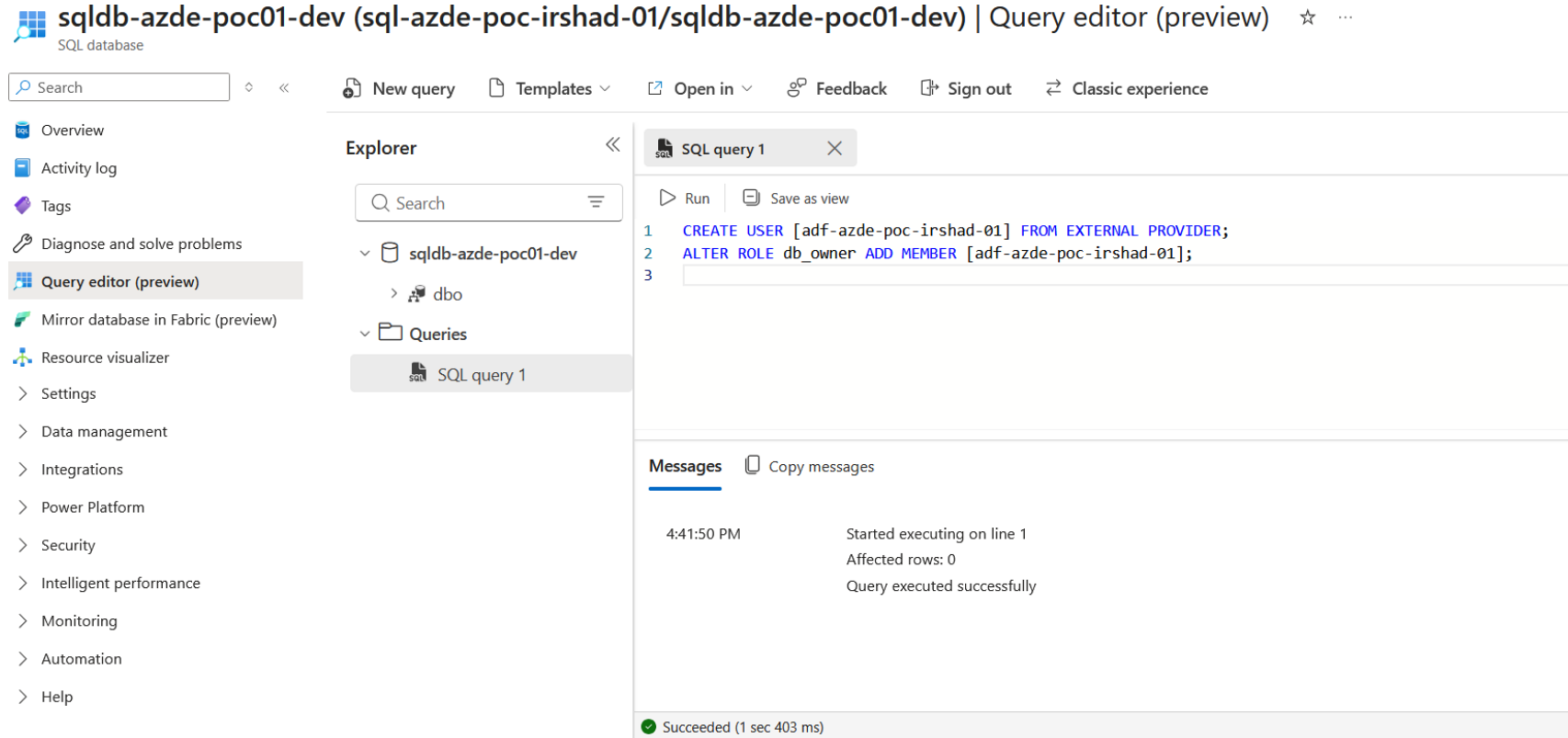
ADF database principal

```
CREATE USER [adf-azde-poc01-<unique>] FROM EXTERNAL PROVIDER;  
ALTER ROLE db_datareader ADD MEMBER [adf-azde-poc01-<unique>];  
ALTER ROLE db_datawriter ADD MEMBER [adf-azde-poc01-<unique>];  
ALTER ROLE db_ddladmin ADD MEMBER [adf-azde-poc01-<unique>];
```

Least-privilege improvement

The manual lab used broad database roles for simplicity. A production design should grant only the exact table/procedure permissions required.

SQL permission evidence



Azure SQL Query Editor used to create the ADF Microsoft Entra database user and grant permissions. What this proves: This allows the ADF Managed Identity to authenticate to the database without a stored SQL password.

6. ADF linked services

LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2. Authentication: system-assigned Managed Identity. Target: POC Storage Account.
LS_AZURE_SQL_MI	Azure SQL Database. Authentication: system-assigned Managed Identity. Target: POC logical server and database.

- Test each linked service before creating datasets.
- For ADLS 403 errors, verify Storage Blob Data Contributor on the ADF identity.
- For SQL failures, verify SQL firewall, Microsoft Entra admin and the contained ADF database user.

ADF linked service - ADLS Gen2

The screenshot shows the 'Linked services' configuration page in the Azure Data Factory portal. The left sidebar contains navigation links: Home, Author, Monitor, Manage, and Learning Center. The 'Manage' section is active, showing a list of linked services. The main panel is titled 'Linked services' and shows a 'New' button and a search filter. The right panel is the configuration form for a new linked service. It includes a title field, a 'Connect via integration runtime' dropdown set to 'AutoResolveIntegrationRuntime', an 'Authentication type' dropdown set to 'Account key', and an 'Account selection method' section with radio buttons for 'From Azure subscription' (selected) and 'Enter manually'. Below this is a dropdown for 'Azure subscription' showing 'Azure subscription 1 (9902e04b-fd65-40f4-86a2-1b09c1fe672c)'. The 'Storage account name' dropdown is set to 'No accounts found' and is highlighted with a red border. At the bottom, there are 'Create', 'Back', 'Test connection', and 'Cancel' buttons.

Home
Author
Monitor
Manage
Learning Center

Data Factory
Validate all
Publish all

General
Connector upgrade advisor
Factory settings
Connections
Linked services
Integration runtimes
Microsoft Purview
ADF in Microsoft Fabric
Source control
Git configuration
ARM template
Author
Triggers
Global parameters
Data flow libraries
Security
Credentials
Customer managed key

Linked services

Linked service defines the connection information to a data store or compute

+ New

Filter by name Annotations : Any

Azure Data Lake Storage Gen2

Connect via integration runtime * ⓘ
AutoResolveIntegrationRuntime

Authentication type
Account key

Account selection method ⓘ
☒ From Azure subscription ☐ Enter manually

Azure subscription ⓘ
Azure subscription 1 (9902e04b-fd65-40f4-86a2-1b09c1fe672c)

Storage account name *
No accounts found

To
☒ To linked service ☐ To file path

Annotations
+ New
> Parameters
> Advanced ⓘ

Create Back Test connection Cancel

Create the ADLS Gen2 linked service with system-assigned Managed Identity.

What this proves: No Storage account key is required in ADF.

ADF linked service list - ADLS

Home

Author

Monitor

Manage

Learning Center

General

Connector upgrade advisor

Factory settings

Connections

Linked services

Integration runtimes

Microsoft Purview

ADF in Microsoft Fabric

Source control

Git configuration

ARM template

Author

Triggers

Global parameters

Data flow libraries

Security

Credentials

Customer managed key

Data Factory

Validate all

Publish all

Migrate to Fabric (Preview)

Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#)

+ New

Filter by name

Annotations : Any

Showing 1 - 1 of 1 items

Name	Type	Related	Annotations
LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2	0	

ADF Manage view showing the ADLS linked service.

What this proves: Test connection should succeed before dataset authoring.

ADF linked services - ADLS and SQL

Home

Author

Monitor

Manage

Learning Center

General

Connector upgrade advisor

Factory settings

Connections

Linked services

Integration runtimes

Microsoft Purview

ADF in Microsoft Fabric

Source control

Git configuration

ARM template

Author

Triggers

Global parameters

Data flow libraries

Security

Credentials

Customer managed key

Data Factory

Validate all

Publish all

Migrate to Fabric (Preview)

Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#)

+ New

Filter by name

Annotations : Any

Showing 1 - 2 of 2 items

Name	Type	Related	Annotations
LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2	0	
LS_AZURE_SQL_MI	Azure SQL Database	0	

Linked-services list after adding Azure SQL connectivity.

What this proves: Both data stores are now available to datasets and activities.

ADF linked service - Azure SQL

Home

Author

Monitor

Manage

Learning Center

Data Factory

Validate all

Publish all

Migrate to Fabric (Preview)

General

Connector upgrade advisor

Factory settings

Connections

Linked services

Integration runtimes

Microsoft Purview

ADF in Microsoft Fabric

Source control

Git configuration

ARM template

Author

Triggers

Global parameters

Data flow libraries

Security

Credentials

Customer managed key

Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#)

+ New

Filter by name

Annotations : Any

Showing 1 - 1 of 1 items

Name	Type	Related	Annotations
LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2	0	

Azure SQL linked-service configuration using Managed Identity.
What this proves: ADF authenticates as its Microsoft Entra identity rather than a SQL password.

7. Create parameterized datasets

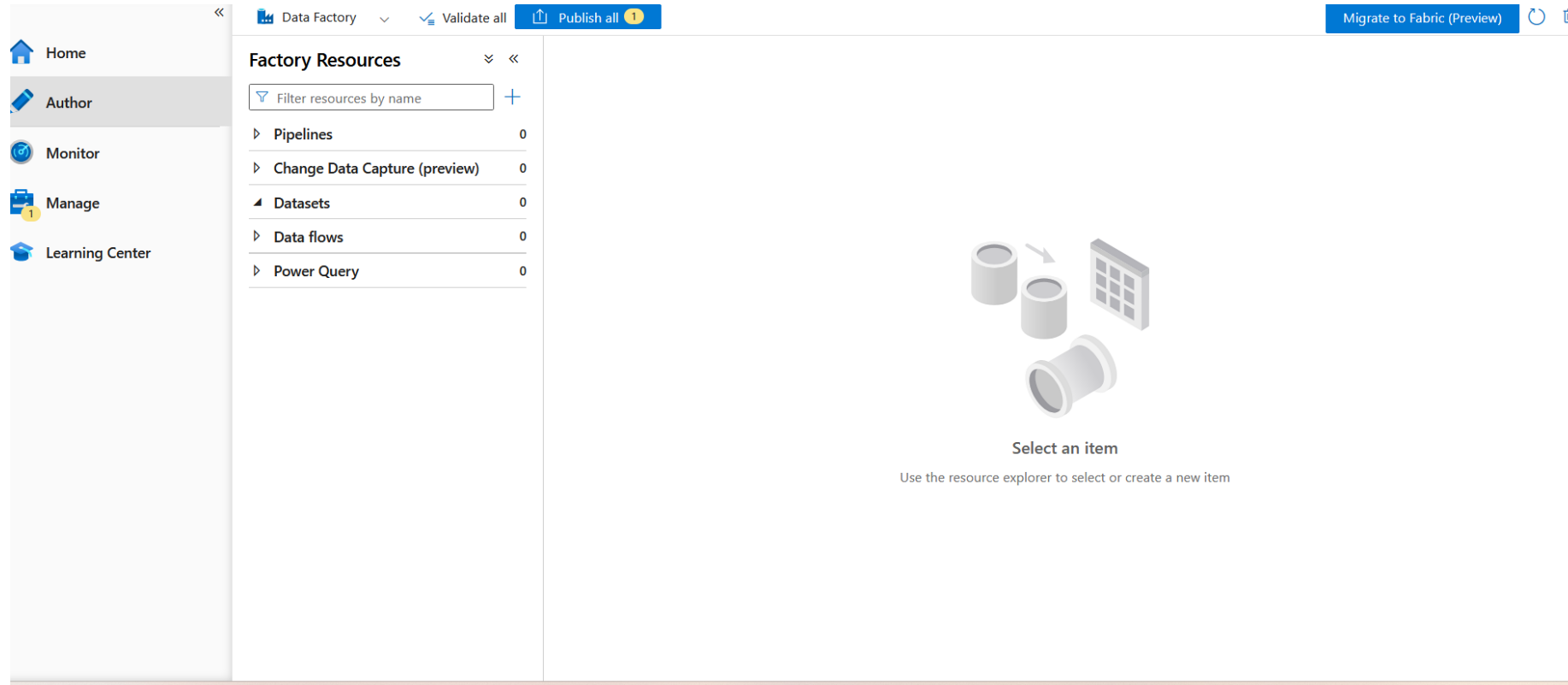
DS_ADLS_OrdersCsv	DelimitedText source. Parameters: p_container, p_folder, p_file. First row as header.
DS_ADLS_Binary	Binary dataset for byte-for-byte archive and Delete Activity. Same three path parameters.
DS_SQL_Table	Azure SQL Database dataset. Parameter: p_target_table. Schema dbo, table from parameter.

Dynamic dataset expressions

```
ADLS dynamic location
File system = @dataset().p_container
Directory   = @dataset().p_folder
File        = @dataset().p_file
```

```
SQL dynamic table
Schema = dbo
Table  = @dataset().p_target_table
```

ADF authoring - begin datasets and pipeline



ADF Studio Author view at the start of dataset/pipeline authoring.

What this proves: The factory has the required authoring workspace and resources.

ADF dataset creation

The screenshot shows the 'Dataset creation properties' dialog box in Azure Data Factory. The left sidebar contains navigation links: Home, Author (selected), Monitor, Manage, and Learning Center. The 'Factory Resources' pane on the left lists: Pipelines (0), Change Data Capture (preview) (0), Datasets (0, selected), Data flows (0), and Power Query (0). The main area displays the following configuration:

- Name:** DS_ADLS_OrdersCsv
- Linked service *:** LS_ADLS_GEN2_MI
- File path:** File system / Directory / File name
- First row as header:** ☒
- Import schema:** ☐ From connection/store, ☐ From sample file, ☒ None

At the bottom right, there are three buttons: OK, Back, and Cancel.

Dataset creation properties during manual authoring.

What this proves: Create the source and sink datasets before wiring pipeline activities.

Configure DS_ADLS_OrdersCsv

Set properties

Name
DS_ADLS_OrdersCsv

Linked service *
LS_ADLS_GEN2_MI

File path
@dataset().p_container / @dataset().p_folder / @dataset().p_file

First row as header ☒

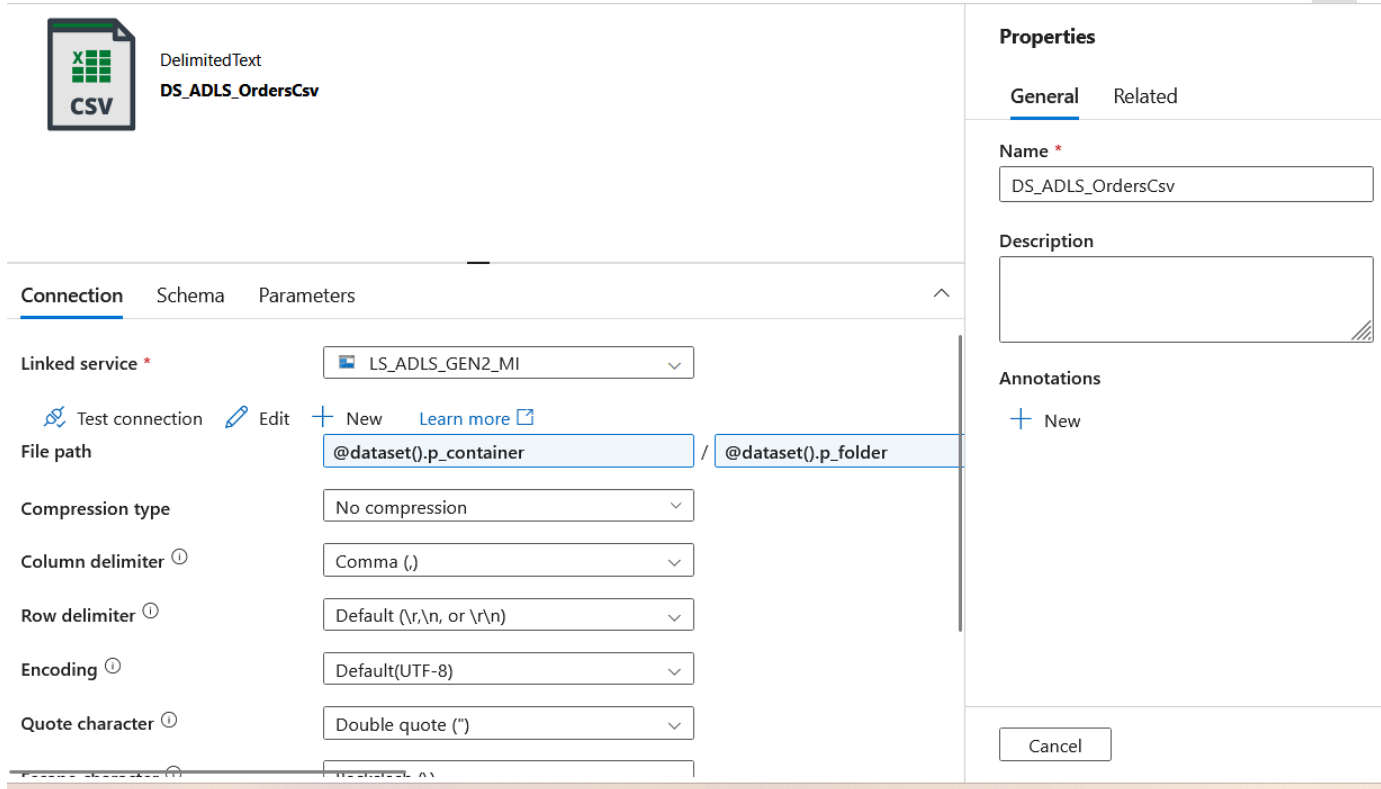
Import schema
☒ From connection/store ☐ From sample file ☐ None

From root
From specified path

Parameterized file-path properties for the ADLS CSV dataset.

What this proves: Pipeline parameters can resolve the same dataset to different folders/files.

DelimitedText connection settings



The screenshot shows the 'DelimitedText' connection settings for a dataset named 'DS_ADLS_OrdersCsv'. The interface is divided into two main sections: the main configuration area on the left and a 'Properties' sidebar on the right.

Main Configuration Area:

- DelimitedText DS_ADLS_OrdersCsv**: Header with a CSV icon.
- Tabs**: 'Connection' (selected), 'Schema', 'Parameters'.
- Linked service ***: A dropdown menu showing 'LS_ADLS_GEN2_MI'.
- Actions**: 'Test connection' (with a test icon), 'Edit' (with a pencil icon), '+ New', and 'Learn more' (with an external link icon).
- File path**: A text field containing '@dataset().p_container' followed by a slash and '@dataset().p_folder'.
- Compression type**: A dropdown menu set to 'No compression'.
- Column delimiter ①**: A dropdown menu set to 'Comma (,)'.
- Row delimiter ①**: A dropdown menu set to 'Default (\r,\n, or \r\n)'.
- Encoding ①**: A dropdown menu set to 'Default(UTF-8)'.
- Quote character ①**: A dropdown menu set to 'Double quote (")'.

Properties Sidebar:

- Properties**: Header with 'General' (selected) and 'Related' tabs.
- Name ***: A text field containing 'DS_ADLS_OrdersCsv'.
- Description**: An empty text area.
- Annotations**: A '+ New' button.
- Cancel**: A button at the bottom.

DelimitedText ADLS orders dataset connection and format settings.

What this proves: First-row-as-header and comma-delimited configuration must match the input file.

Configure the Binary ADLS dataset

The screenshot displays the Azure Data Studio interface for configuring a Binary ADLS dataset. On the left, a sidebar shows a tree view with 'Pipelines' (0), 'Change Data Capture (preview)' (0), 'Datasets' (2), and 'Data flows' (0). Under 'Datasets', 'DS_ADLS_Binary' is selected. The main workspace shows a 'Binary' dataset named 'DS_ADLS_Binary' with a '01' icon. Below this, the 'Connection' tab is active, showing a 'Linked service' dropdown set to 'LS_ADLS_GEN2_MI'. A 'Test connection' button with a green checkmark indicates 'Connection successful'. The 'File path' is set to '@dataset().p_container' / '@dataset().p_folder', and the 'Compression type' is 'No compression'. On the right, the 'Properties' panel shows the 'General' tab with the 'Name' field set to 'DS_ADLS_Binary' and a 'Description' field. A 'Cancel' button is at the bottom right.

Properties

General Related

Name *

DS_ADLS_Binary

Description

Annotations

+ New

Cancel

Binary ADLS dataset settings for archive/delete operations.

What this proves: Binary copy preserves the raw source bytes without interpreting CSV columns.

Configure Azure SQL connectivity in ADF

The screenshot displays the Azure Data Factory (ADF) interface. On the left, a navigation pane shows the 'Datasets' section with three items: 'DS_ADLS_Binary', 'DS_ADLS_OrdersCsv', and 'DS_SQL_Table'. The 'DS_SQL_Table' dataset is selected. The main area shows the 'Connection' tab for the 'DS_SQL_Table' dataset. The 'Linked service' is 'LS_AZURE_SQL_MI'. Below this, there are options to 'Test connection', 'Edit', 'New', and 'Learn more'. The 'Table' dropdown is set to 'Select...'. The 'Enter manually' checkbox is unchecked. The 'Preview data' button is visible. On the right, the configuration details for the linked service are shown. The 'Description' field is empty. The 'Connect via integration runtime' dropdown is set to 'AutoResolveIntegrationRuntime'. The 'Version' dropdown is set to '2.0 (Recommended)'. The 'Account selection method' is set to 'Enter manually'. The 'Fully qualified domain name' is 'sql-azde-poc-irshad-01.database.windows.net'. The 'Database name' is 'sqldb-azde-poc01-dev'. The 'Authentication type' is 'System-assigned managed identity'. The 'Managed identity name' is 'adf-azde-poc-irshad-01'. At the bottom, there are 'Save' and 'Cancel' buttons, and a 'Test connection' button.

Filter resources by name +

- Pipelines 0
- Change Data Capture (preview) 0
- Datasets 3**
 - DS_ADLS_Binary
 - DS_ADLS_OrdersCsv
 - DS_SQL_Table**
- Data flows 0
- Power Query 0

Azure SQL Database
DS_SQL_Table

Connection Schema Parameters

Linked service * LS_AZURE_SQL_MI

Test connection Edit + New Learn more

Table Select... Enter manually

Preview data

Description

Connect via integration runtime * ⓘ
AutoResolveIntegrationRuntime

Version
☒ 2.0 (Recommended) ☐ 1.0

Account selection method ⓘ
☐ From Azure subscription ☒ Enter manually

Fully qualified domain name *
sql-azde-poc-irshad-01.database.windows.net

Database name *
sqldb-azde-poc01-dev

Authentication type *
System-assigned managed identity

Managed identity name: adf-azde-poc-irshad-01

Save Cancel Test connection

Azure SQL linked-service configuration using Managed Identity.

What this proves: This connection is reused by the SQL dataset and Script/Stored Procedure activities.

Configure DS_SQL_Table

The screenshot displays the Azure Data Factory Author interface. On the left, a navigation pane includes links for Home, Author (3), Monitor, Manage (1), and Learning Center. The main area is titled 'Factory Resources' and contains a search bar and a list of resources: Pipelines (0), Change Data Capture (preview) (0), Datasets (3), and Data flows (0). Under Datasets, three items are listed: DS_ADLS_Binary, DS_ADLS_OrdersCsv, and DS_SQL_Table (selected). The right pane shows the configuration for DS_SQL_Table, which is an Azure SQL Database dataset. The 'Connection' tab is active, showing the linked service 'LS_AZURE_SQL_MI' and a successful connection status. The 'Table' field is set to 'dbo', and the 'Enter manually' checkbox is checked. The 'Parameters' tab is also visible, showing a parameter '@dataset().p_target_table'. The 'Properties' pane on the right shows the 'General' tab with the Name 'DS_SQL_Table' and a Description field. A 'Cancel' button is located at the bottom right.

Parameterized Azure SQL table dataset.

What this proves: The pipeline passes orders_stg as p_target_table.

8. Build PL_INGEST_ORDERS_BATCH

p_container	String, default landing
p_folder	String, default orders/2026/08/28
p_file	String, default orders_001.csv
p_target_table	String, default orders_stg

Activity order

- 1. Get_Metadata_File - check Exists, Size and Last modified.
- 2. If_File_Exists - True continues; False goes to Fail_File_Not_Found.
- 3. Script_Log_Start - insert IN_PROGRESS row.
- 4. Copy_Orders_To_Staging - CSV to dbo.orders_stg; truncate staging before copy.
- 5. SP_Merge_Orders - execute dbo.usp_merge_orders.
- 6. Copy_Archive_File - landing to archive using Binary dataset.
- 7. Delete_Landing_File - remove processed landing copy.
- 8. Script_Log_Success - update the ETL log to SUCCESS.

Pipeline expressions and SQL snippets

If condition, failure message and pre-copy

```
If condition
@activity('Get_Metadata_File').output.exists

Fail message
@concat('File does not exist: ', pipeline().parameters.p_container, '/',
        pipeline().parameters.p_folder, '/', pipeline().parameters.p_file)

Copy pre-copy script
TRUNCATE TABLE dbo.orders_stg;
```

Script_Log_Start

```
Script_Log_Start
INSERT INTO dbo.etl_file_log (file_name, file_size_bytes, status, started_at)
VALUES ('@{pipeline().parameters.p_file}',
        @{activity('Get_Metadata_File').output.size},
        'IN_PROGRESS', SYSUTCDATETIME());
```

Pipeline success logging

Script_Log_Success

```
UPDATE dbo.etl_file_log
SET status = 'SUCCESS',
    rows_inserted = (SELECT COUNT(1) FROM dbo.orders
                     WHERE source_file = '{@pipeline().parameters.p_file}'),
    rows_updated = 0,
    completed_at = SYSUTCDATETIME()
WHERE file_name = '{@pipeline().parameters.p_file}'
AND status = 'IN_PROGRESS';
```

Dependency rule

Each downstream activity must depend on the successful completion of the preceding required activity. Archive only after MERGE succeeds; delete landing only after archive succeeds.

Pipeline parameters

The screenshot displays the Azure Data Factory Author interface. On the left is a navigation pane with links to Home, Author, Monitor, Manage, and Learning Center. The main workspace is divided into several sections:

- Factory Resources:** A tree view showing the hierarchy of resources. Under 'Pipelines', the 'PL_INGEST_ORDERS_BATCH' pipeline is selected.
- Activities:** A list of available activities including Move and transform, Synapse, Azure Data Explorer, Azure Function, Batch Service, Databricks, General, HDInsight, Iteration & conditionals, Machine Learning, and Power Query.
- Parameters:** A table listing the parameters for the selected pipeline. The table has columns for Name, Type, and Default value.
- Properties:** A panel on the right showing the general properties of the pipeline, including its Name and Description.

The 'Parameters' table is as follows:

Name	Type	Default value
p_container	String	landing
p_folder	String	rs/2026/08/28
p_file	String	orders_001.csv
p_target_table	String	orders_stg

PL_INGEST_ORDERS_BATCH pipeline parameters.

What this proves: The default values point to the manual landing file and staging table.

Get Metadata activity

The screenshot displays the Microsoft Fabric Data Factory Author interface. On the left is a navigation pane with links to Home, Author (highlighted), Monitor, Manage, and Learning Center. The main workspace is divided into several sections:

- Factory Resources:** A list of resources including Pipelines (1), Datasets (3), Data flows (0), and Power Query (0). The selected pipeline is 'PL_INGEST_ORDERS_BATCH'.
- Activities:** A list of available activities such as Delete, Execute Pipeline, Execute SSIS package, Fail, Get Metadata (selected), Lookup, Stored procedure, Script, Set variable, Validation, Web, WebHook, and Wait.
- Activity Configuration:** The 'Get Metadata' activity is configured with the following settings:
 - Dataset:** DS_ADLS_OrdersCsv
 - Dataset properties:** A table listing properties for the dataset.
 - Field list:** A section for defining fields to retrieve metadata.
- Properties:** A panel on the right showing the activity's properties, including Name (PL_INGEST_ORDERS_BATCH) and Description.

Get_Metadata_File activity configuration.

What this proves: It reads the parameterized CSV dataset.

Get Metadata fields

The screenshot displays the Microsoft Fabric Data Factory Author interface. On the left is a navigation pane with 'Home', 'Author' (selected), 'Monitor', 'Manage', and 'Learning Center'. The main area is divided into three panes: 'Factory Resources', 'Activities', and the 'Get Metadata' activity configuration pane.

Factory Resources: Shows a list of resources including Pipelines (PL_INGEST_ORDERS_BATCH), Change Data Capture (preview), and Datasets (DS_ADLS_Binary, DS_ADLS_OrdersCsv, DS_SQL_Table).

Activities: A list of available activities including Execute Pipeline, Execute SSIS package, Fail, Get Metadata, Lookup, Stored procedure, Script, Set variable, Validation, Web, WebHook, and Wait.

Get Metadata Activity Configuration: The 'Settings' tab is active. It shows the 'p_file' parameter set to '@pipeline().parameters.p_file' (string). Below this is the 'Field list' section with a 'New' button and a 'Delete' button. The 'Field list' contains three items, each with a checkbox and a dropdown menu:

- ☐ Argument
- ☐ Exists
- ☐ Size
- ☐ Last modified

At the bottom, there are filters for 'Filter by last modified' (with a help icon), 'Start time (UTC)', 'End time (UTC)', and 'Skip line count'.

Get Metadata field-list settings.

What this proves: Exists, Size and Last modified provide both control and audit information.

If_File_Exists activity

The screenshot displays the Azure Data Factory (ADF) designer interface. On the left, the 'Factory Resources' pane shows a list of resources including Pipelines (PL_INGEST_ORDERS_BATCH), Change Data Capture (preview), and Datasets (DS_ADLS_Binary, DS_ADLS_OrdersCsv, DS_SQL_Table). The 'Activities' pane on the right shows a search for 'if' and a list of activities including 'If Condition'. The main canvas shows a workflow with two activities: 'Get Metadata' (labeled 'Get_Metadata_File') and 'If Condition' (labeled 'If_File_Exists'). A green arrow connects the output of 'Get Metadata' to the 'If Condition' activity. The 'If Condition' activity is expanded, showing its configuration. The 'Expression' field is set to '@activity('Get_Metadata_File').output...'. Below this, the 'Case' section shows two rows: 'True' and 'False', both with the value 'No activities' and a pencil icon for editing.

Factory Resources

Filter resources by name

Pipelines 1

- PL_INGEST_ORDERS_BATCH

Change Data Capture (preview) 0

Datasets 3

- DS_ADLS_Binary
- DS_ADLS_OrdersCsv
- DS_SQL_Table

Data flows 0

Power Query 0

Activities

if

Iteration & conditionals

- If Condition

Get Metadata

Get_Metadata_File

If Condition

If_File_Exists

Expression ⓘ

@activity('Get_Metadata_File').output...

Case	Activity
True	No activities
False	No activities

If condition connected after Get Metadata.

What this proves: The pipeline explicitly branches on file existence.

Fail_File_Not_Found branch

The screenshot displays the Azure Data Factory Author interface. On the left, the navigation pane includes Home, Author, Monitor, Manage, and Learning Center. The 'Factory Resources' pane on the right shows a list of resources: Pipelines (1), Datasets (3), Data flows (0), and Power Query (0). The selected pipeline is 'PL_INGEST_ORDERS_BATCH'. The 'Activities' pane shows a search for 'fa' and a list of activities including 'Fail'. The main canvas shows a workflow starting with a 'Get Metadata' activity (labeled 'Get_Metadata_File') that feeds into an 'If Condition' activity (labeled 'If_File_Exists'). The 'If Condition' activity has two branches: 'True' and 'False'. The 'True' branch leads to a connector (+). The 'False' branch leads to a 'Fail_File_Not_Found' activity, which then leads to another connector (+). The bottom pane shows the 'Activities (1)' tab with the following table:

Case	Activity
True	No activities
False	Fail_File_Not_Fo...

The 'Expression' tab shows the expression: `@activity('Get_Metadata_File').output...`.

False branch with a Fail activity for missing input.

What this proves: A missing source is treated as a controlled failure instead of a silent success.

Lookup/query authoring evidence

The screenshot displays the Microsoft Fabric Data Factory Authoring interface. On the left, a navigation pane includes 'Home', 'Author' (selected), 'Monitor', 'Manage', and 'Learning Center'. The 'Factory Resources' pane shows a tree view with 'Pipelines' (1 item: PL_INGEST_ORDERS_BATCH) and 'Datasets' (3 items: DS_ADLS_Binary, DS_ADLS_OrdersCsv, DS_SQL_Table). The 'Activities' pane shows a search for 'lookup' with a 'Lookup' activity selected. The main canvas shows the 'PL_INGEST_ORDERS_BATCH' pipeline with an 'If_File_Exists' trigger leading to 'True activities'. A 'Lookup' activity is configured with the following settings:

Name	Value	Type
p_target_table	etl_file_log	string

Additional settings for the Lookup activity:

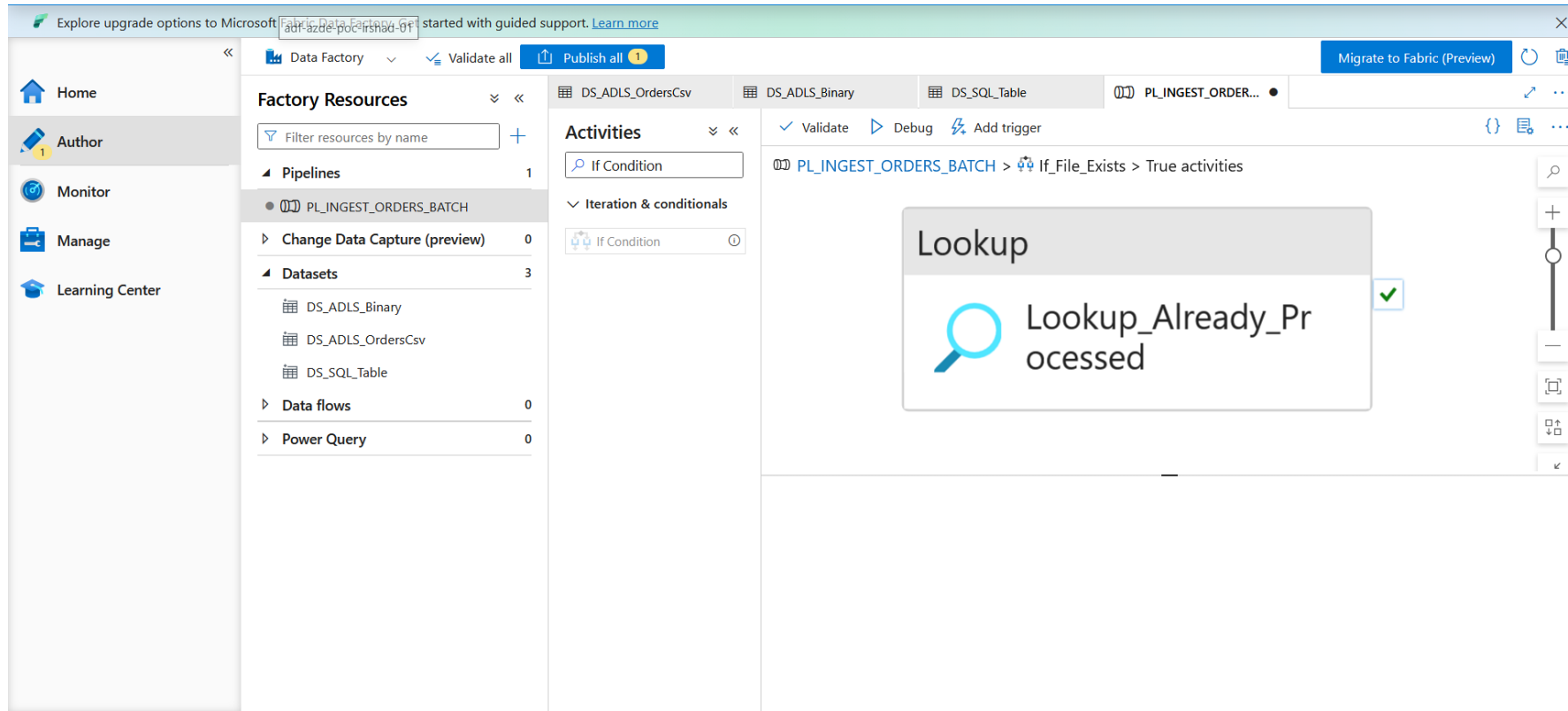
- First row only: ☒
- Use query: ☒ Table ☐ Query ☐ Stored procedure
- Query: `SELECT CASE WHEN EXISTS (SELECT ...`
- Query timeout (minutes): 120
- Isolation level: Select...
- Partition option: ☒ None ☐ Physical partitions of table ☐ Dynamic range

A message at the bottom states: 'Please preview data to validate the partition settings.'

ADF lookup/query configuration used during pipeline authoring.

What this proves: The repository advanced variant uses lookup-based processed-file control; the manual pipeline can remain simpler.

Lookup activity on pipeline canvas



Lookup activity visible on the ADF pipeline canvas.

What this proves: This evidence belongs to the enhanced file-control authoring path.

Copy CSV to SQL staging

The screenshot displays the Microsoft Fabric Data Factory Author interface. On the left, the navigation pane includes Home, Author, Monitor, Manage, and Learning Center. The main workspace is divided into several sections:

- Factory Resources:** A list of resources including Pipelines (PL_INGEST_ORDERS_BATCH), Datasets (DS_ADLS_Binary, DS_ADLS_OrdersCsv, DS_SQL_Table), Data flows, and Power Query.
- Activities:** A search bar and a list of activities, with 'Copy data' selected.
- Copy data activity configuration:** The 'Sink' tab is active, showing the following settings:
 - General:** Name: p_target_table, Value: orders_stg, Type: string.
 - Write behavior:** Insert (selected), Upsert, Stored procedure.
 - Bulk insert table lock:** Yes, No (selected).
 - Table option:** Use existing (selected), Auto create table.
 - Pre-copy script:** TRUNCATE TABLE ~~dbo.orders_stg~~;
 - Write batch timeout:** e.g. 00:30:00.
 - Write batch size:** A dropdown menu.

The top of the interface shows a banner for Microsoft Fabric Data Factory, a 'Publish all' button, and a 'Migrate to Fabric (Preview)' button. The bottom of the workspace shows a diagram of the data flow, including a 'Get Metadata' activity, an 'If File Exists' condition, a 'Script' activity, and the 'Copy data' activity.

Copy Activity configuration including staging/pre-copy behavior.

What this proves: TRUNCATE before copy makes each manual load start with a clean staging table.

ADF Script activity

The screenshot displays the Microsoft Azure Data Factory (ADF) Author interface. The left sidebar contains navigation options: Home, Author (selected), Monitor, Manage, and Learning Center. The main workspace is divided into three panes. The 'Factory Resources' pane on the left lists 'Pipelines' (1) and 'Datasets' (3). The 'Activities' pane in the center shows a 'Script' activity selected. The right pane displays the pipeline canvas with a sequence of activities: 'Get Metadata', 'If Condition', 'Script' (highlighted), 'Copy data', 'Stored procedure', 'Copy data', 'Delete', and 'Script' (highlighted). Below the canvas, the 'Settings' tab is active, showing the 'Linked service' as 'LS_AZURE_SQL_MI' and the 'Script' type as 'Query'. The script text is 'UPDATE dbo.etl_file_log SET status = ...'.

Script/SQL activity configuration in the pipeline.

What this proves: SQL logging is performed as part of orchestration.

Initial pipeline debug

The screenshot displays the Microsoft Fabric Data Factory Author interface. The left sidebar contains navigation links: Home, Author, Monitor, Manage, and Learning Center. The main workspace is divided into several panels:

- Factory Resources:** A list of resources including Pipelines (1), Datasets (3), Data flows (0), and Power Query (0). The selected pipeline is `PL_INGEST_ORDERS_BATCH`.
- Activities:** A search bar and a list of activities, with `Script` selected.
- Pipeline Design View:** A visual representation of the pipeline flow. It starts with an `If File Exists` condition, followed by a `Script` activity (labeled `Script_Log_Start`), a `Copy data` activity (labeled `Copy_Orders_To_Staging`), a `Stored procedure` activity (labeled `SP_Merge_Orders`), another `Copy data` activity (labeled `Copy_Archive_File`), and finally a `Delete` activity (labeled `Delete_Landing_File`).
- Output Panel:** A tabbed interface showing the results of the pipeline run. The `Output` tab is active, displaying the following information:

Pipeline run ID: `c1d9ffb0-6db6-48c2-bb19-a88e40853425`

Pipeline status: Succeeded (indicated by a green checkmark icon)

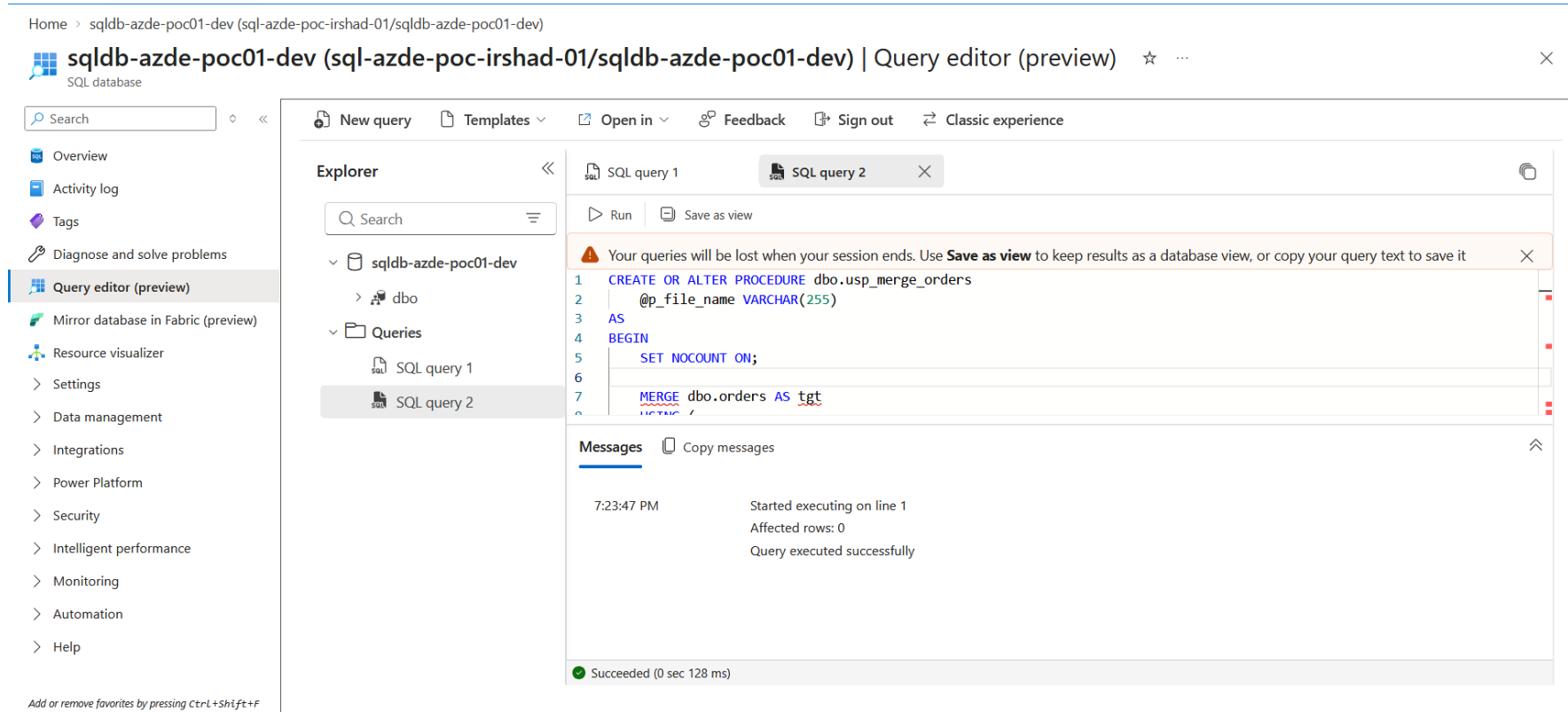
Activity-level outputs:

Activity name	Activity status	Activity type	Run start	Duration	Integration
If_File_Exists	Succeeded	If Condition	8/28/2026, 6:33:29 PM	2s	

ADF pipeline debug/output during early execution.

What this proves: Use activity-level outputs to locate the first failing step.

SQL Query Editor during testing



Azure SQL Query Editor used while configuring and testing database objects.

What this proves: SQL-side verification is independent evidence that the pipeline did the expected work.

ADLS container verification

Home > Storage center | Blob Storage > stazdepocirshad01

Storage center | Blob Storage

Default Directory (erirshad26outlook.onmicrosoft.com)

Search

SummaryResources

Overview

All storage resources

Object storage

File storage

Block storage

Data management

Migration

Partner solutions

Management services

Help

CreateRestore...

Name ↑

stazdepocirshad01

Showing 1 - 1 of 1. Display auto

count:

Add or remove favorites by pressing Ctrl+Shift+F

stazdepocirshad01 | Containers

Storage account

Search

Add containerUploadRefreshDeleteChange access level...

Search containers by prefix

Only show active containers

Showing all 1 items

Name	Last modified	Anonymous access level	Lease state
\$logs	28/8/2026, 3:39:11 pm	Private	Available

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

Data storage

Containers

Classic file shares

Queues

Tables

Security + networking

Data management

Add or remove favorites by pressing Ctrl+Shift+F

Storage/container verification during pipeline testing.

What this proves: Check landing/archive paths directly instead of relying only on pipeline status.

Get Metadata execution evidence

The screenshot displays the Microsoft Fabric Data Factory interface. On the left, the 'Factory Resources' pane shows a list of resources under 'Pipelines', with 'PL_INGEST_ORDERS_BATCH' selected. The main canvas shows a pipeline named 'PL_INGEST_ORDER...' with an 'If Condition' activity containing an 'If File Exists' trigger. Below the canvas, the 'Output' tab is active, showing the 'Get Metadata' activity's execution details. The activity is marked as 'Succeeded' with a green checkmark. The pipeline run ID is '2b5001d5-e489-4b82-b6a6-b4c5b90e0e24' and the status is 'In progress'.

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime
Get_Metadata_File	✓ Succeeded	Get Metadata	8/28/2026, 7:32:39 PM	30s	AutoResolveIntegrationRuntime (East US)

Get_Metadata_File succeeded during a pipeline run.

What this proves: A green activity confirms that ADF can reach the configured source path.

9. Python authentication - understand the az error

The Poetry packages `azure-identity` and `azure-storage-file-datalake` are Python SDK libraries. They do not install the `az` command. Azure CLI is a separate Windows application.

Install and verify Azure CLI

```
winget install --exact --id Microsoft.AzureCLI

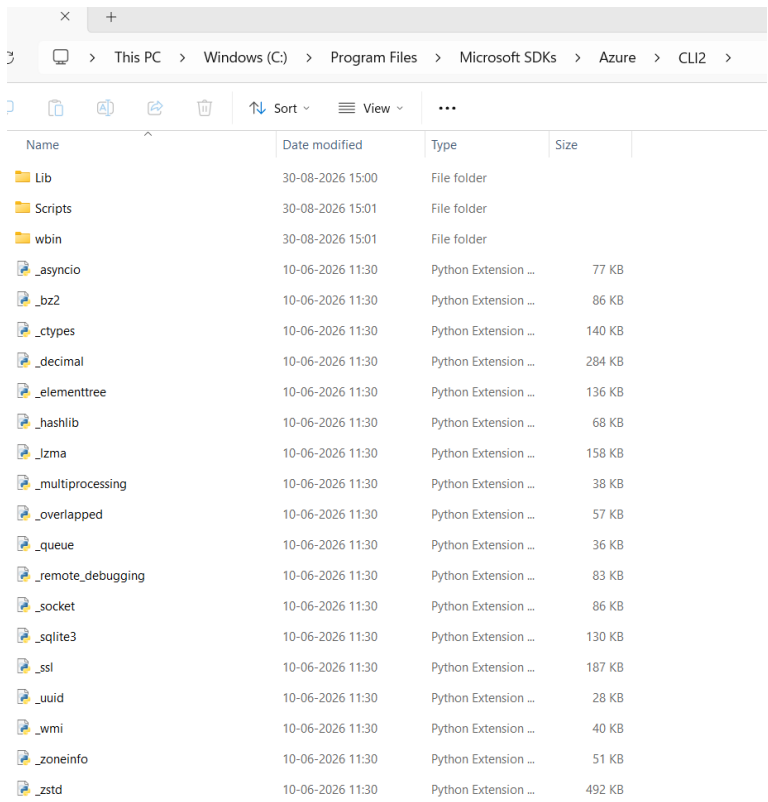
# Close all CMD/PowerShell windows, then open a new terminal
az version
az login
az account show -o table
az group show --name rg-azde-poc01-dev -o table

# If PATH still looks wrong
where az
```

Observed error: 'az' is not recognized

Install Microsoft Azure CLI separately. After installation, close all terminals and reopen a new CMD/PowerShell so Windows reloads PATH. Verify with `az version` before attempting `az login`.

Azure CLI installation evidence



The screenshot shows a Windows File Explorer window with the address bar displaying the path: This PC > Windows (C:) > Program Files > Microsoft SDKs > Azure > CLI2. The main pane shows a list of files and folders. The 'Name' column lists folders like 'Lib', 'Scripts', and 'wbin', followed by numerous Python extension modules such as '_asyncio', '_bz2', '_ctypes', '_decimal', '_elementtree', '_hashlib', '_lzma', '_multiprocessing', '_overlapped', '_queue', '_remote_debugging', '_socket', '_sqlite3', '_ssl', '_uuid', '_wmi', '_zoneinfo', and '_zstd'. The 'Date modified' column shows dates for folders (30-08-2026 15:00) and extensions (10-06-2026 11:30). The 'Type' column identifies folders and Python extensions. The 'Size' column shows sizes for the extension files, ranging from 28 KB to 492 KB.

Name	Date modified	Type	Size
Lib	30-08-2026 15:00	File folder	
Scripts	30-08-2026 15:01	File folder	
wbin	30-08-2026 15:01	File folder	
_asyncio	10-06-2026 11:30	Python Extension ...	77 KB
_bz2	10-06-2026 11:30	Python Extension ...	86 KB
_ctypes	10-06-2026 11:30	Python Extension ...	140 KB
_decimal	10-06-2026 11:30	Python Extension ...	284 KB
_elementtree	10-06-2026 11:30	Python Extension ...	136 KB
_hashlib	10-06-2026 11:30	Python Extension ...	68 KB
_lzma	10-06-2026 11:30	Python Extension ...	158 KB
_multiprocessing	10-06-2026 11:30	Python Extension ...	38 KB
_overlapped	10-06-2026 11:30	Python Extension ...	57 KB
_queue	10-06-2026 11:30	Python Extension ...	36 KB
_remote_debugging	10-06-2026 11:30	Python Extension ...	83 KB
_socket	10-06-2026 11:30	Python Extension ...	86 KB
_sqlite3	10-06-2026 11:30	Python Extension ...	130 KB
_ssl	10-06-2026 11:30	Python Extension ...	187 KB
_uuid	10-06-2026 11:30	Python Extension ...	28 KB
_wmi	10-06-2026 11:30	Python Extension ...	40 KB
_zoneinfo	10-06-2026 11:30	Python Extension ...	51 KB
_zstd	10-06-2026 11:30	Python Extension ...	492 KB

Windows Azure CLI installation directory after installing the CLI.

What this proves: This proves Azure CLI is a separate local installation from the Poetry Python packages.

Python environment and ADLS upload

Verify the Poetry environment

```
poetry run python --version  
poetry run python -c "import azure.identity; import azure.storage.filedatalake; print('Azure Python SDK OK')"
```

The completed manual pipeline expects this 5-row order_date CSV:

Manual 5-row CSV

```
order_id,customer_id,order_date,product_id,quantity,unit_price,status  
1001,501,2026-08-28,10,2,25.50,Completed  
1002,502,2026-08-28,12,1,100.00,Pending  
1003,503,2026-08-28,15,4,15.75,Completed  
1004,504,2026-08-28,10,1,25.50,Shipped  
1005,505,2026-08-28,20,3,45.00,Completed
```

Upload the manual CSV to ADLS

Upload command

```
poetry run python python\upload_to_adls.py ^  
  --storage-account <your-storage-account> ^  
  --container landing ^  
  --remote-path orders/2026/08/28/orders_001.csv ^  
  --local-file data/generated/orders_001.csv ^  
  --overwrite
```

Authentication flow

az login -> Microsoft Entra token -> DefaultAzureCredential -> DataLakeServiceClient -> ADLS Gen2. No Storage key or SAS token is required in the script.

If Python returns 403

Assign Storage Blob Data Contributor to the signed-in developer user. A role assigned only to the ADF Managed Identity does not automatically authorize your local Python process.

Developer RBAC and SQL verification

The image displays a SQL query editor interface for a database named 'sqldb-azde-poc01-dev'. The interface includes a sidebar with navigation options like Overview, Activity log, Tags, and Query editor (preview). The main area shows a query editor with a single query: `SELECT COUNT(*) AS total_curated_orders FROM dbo.orders;`. The results tab shows a single result: `123 total_curated_orders`. Below the query editor, a terminal window is open, displaying Azure CLI commands and their output. The commands are: `USER_ID = az ad signed-in-user show --query id -o tsv`, `STORAGE_ID = az storage account show --name stazdepocirshad01 --query id -o tsv`, and `az role assignment create --assignee 'Blob Data Contributor' --role 'Storage Blob Data Contributor' --scope 'https://storage.azure.com/' --principal-id $USER_ID --principal-type User`. The output shows the user ID, storage account ID, and the role assignment details.

SQL database: sqldb-azde-poc01-dev (sqi-azde-poc-irsnad-01/sqldb-azde-poc01-dev) | Query editor (preview)

Search

New query Templates Open in Feedback Sign out Classic experience

Overview Activity log Tags Diagnose and solve problems Query editor (preview) Mirror database in Fabric (preview) Resource visualizer

Explorer

Search

sqldb-azde-poc01-dev

dbo

Queries

SQL query 1

SQL query 1

Run Save as view

1 SELECT COUNT(*) AS total_curated_orders FROM dbo.orders;

Messages Results

123 total_curated_orders

Restart Manage files New session Editor Open in VS Code for the Web Web preview Settings Help

```
USER_ID = az ad signed-in-user show --query id -o tsv
STORAGE_ID = az storage account show --name stazdepocirshad01 --query id -o tsv
az role assignment create \
  --assignee "Blob Data Contributor" \
  --role "Storage Blob Data Contributor" \
  --scope "https://storage.azure.com/" \
  --principal-id $USER_ID \
  --principal-type User
{
  "id": "123",
  "name": "123",
  "type": "User",
  "roles": [
    {
      "role_name": "Storage Blob Data Contributor",
      "role_id": "123"
    }
  ],
  "permissions": {
    "actions": [
      "Storage Blob Data Contributor"
    ],
    "data_actions": [
      "Storage Blob Data Contributor"
    ],
    "data_actions_ex": [
      "Storage Blob Data Contributor"
    ],
    "actions_ex": [
      "Storage Blob Data Contributor"
    ],
    "data_actions_ex": [
      "Storage Blob Data Contributor"
    ],
    "actions_ex": [
      "Storage Blob Data Contributor"
    ]
  },
  "created_on": "2026-08-30T11:09:36.729082+00:00",
  "identity_resource_id": null,
  "principal_id": "123",
  "principal_name": "123",
  "principal_type": "User",
  "role_name": "Storage Blob Data Contributor",
  "role_id": "123"
}
```

Azure CLI/Cloud Shell role-assignment evidence together with SQL verification.

What this proves: The developer identity receives Storage Blob Data Contributor separately from ADF.

10. End-to-end run and verification

- Confirm orders_001.csv exists in landing/orders/2026/08/28/.
- ADF Studio -> Validate all.
- Publish all.
- Run Debug or Trigger now using the default parameters.
- Open Monitor and inspect each activity.
- Run SQL verification queries.
- Verify the raw file exists in archive and no longer exists in landing.

Verification queries

```
SELECT * FROM dbo.orders ORDER BY order_id;  
SELECT COUNT(*) AS total_curated_orders FROM dbo.orders;  
SELECT * FROM dbo.etl_file_log ORDER BY log_id DESC;
```

Pass condition

dbo.orders contains exactly 5 business rows and the latest ETL log row is SUCCESS.

Controlled failure - file not found

The screenshot displays the Microsoft Fabric Data Factory Author interface. On the left, the 'Factory Resources' pane shows a tree view with 'Pipelines' expanded, listing 'PL_INGEST_ORDERS_BATCH'. The main canvas shows a pipeline diagram with three activities: 'Get Metadata' (labeled 'Get_Metadata_File'), an 'If' condition, and a 'Script' activity (labeled 'Script_Log_Start'). The 'If' condition has two branches: 'True' leading to the 'Script' activity, and 'False' leading to a 'Fail' activity labeled 'Fail_File_Not_Found'. The 'Pipeline status' at the bottom indicates the run failed. Below the status, a table lists the activities and their outcomes.

Activity name	Activity st...	Activit...	Run start	Duration	Integration
Fail_File_Not_Found	Failed	Fail	8/30/2026, 10:26:32 AM	Less than 1s	
If_File_Exists	Failed	If Condition	8/30/2026, 10:26:32 AM	2s	
Get_Metadata_File	Succeeded	Get Metadata	8/30/2026, 10:26:13 AM	18s	AutoResolve

Get Metadata succeeded, but the existence condition followed the false branch and Fail_File_Not_Found failed the run.

What this proves: This is expected when the landing file is absent.

Issue and fix

Re-upload the file to the exact landing/orders/2026/08/28/orders_001.csv path, or correct p_folder/p_file. Then rerun.

Troubleshooting - Script_Log_Start failed

The screenshot displays the Microsoft Fabric Data Factory Author interface. On the left, the 'Factory Resources' pane shows a tree view with 'Pipelines' expanded, listing 'PL_INGEST_ORDERS_BATCH'. The main canvas shows a pipeline diagram with three activities: 'If_File_Exists' (succeeded), 'Script' (failed), and 'Copy data' (succeeded). The 'Script' activity is highlighted with a red 'X' icon. Below the canvas, the 'Output' tab shows the pipeline run details for 'PL_INGEST_ORDERS_BATCH' (run ID: 1-43018636b08c). The pipeline status is 'Failed'. A table below lists the activities and their status:

Activity name	Activity st...	Activit...	Run start	Duration	Integration
Script_Log_Start	Failed	Script	8/30/2026, 10:52:42 AM	1m 15s	AutoResolve
If_File_Exists	Succeeded	If Condition	8/30/2026, 10:52:40 AM	Less than 1s	
Get_Metadata_File	Succeeded	Get Metadata	8/30/2026, 10:52:31 AM	8s	AutoResolve

The file check passed, but Script_Log_Start failed after about 1 minute 15 seconds.

What this proves: This isolates the problem to Azure SQL connectivity/authentication rather than ADLS.

Issue and fix

Check the SQL logical-server firewall and enable the beginner-lab Azure services exception. Also verify Microsoft Entra admin and the ADF contained database user/permissions.

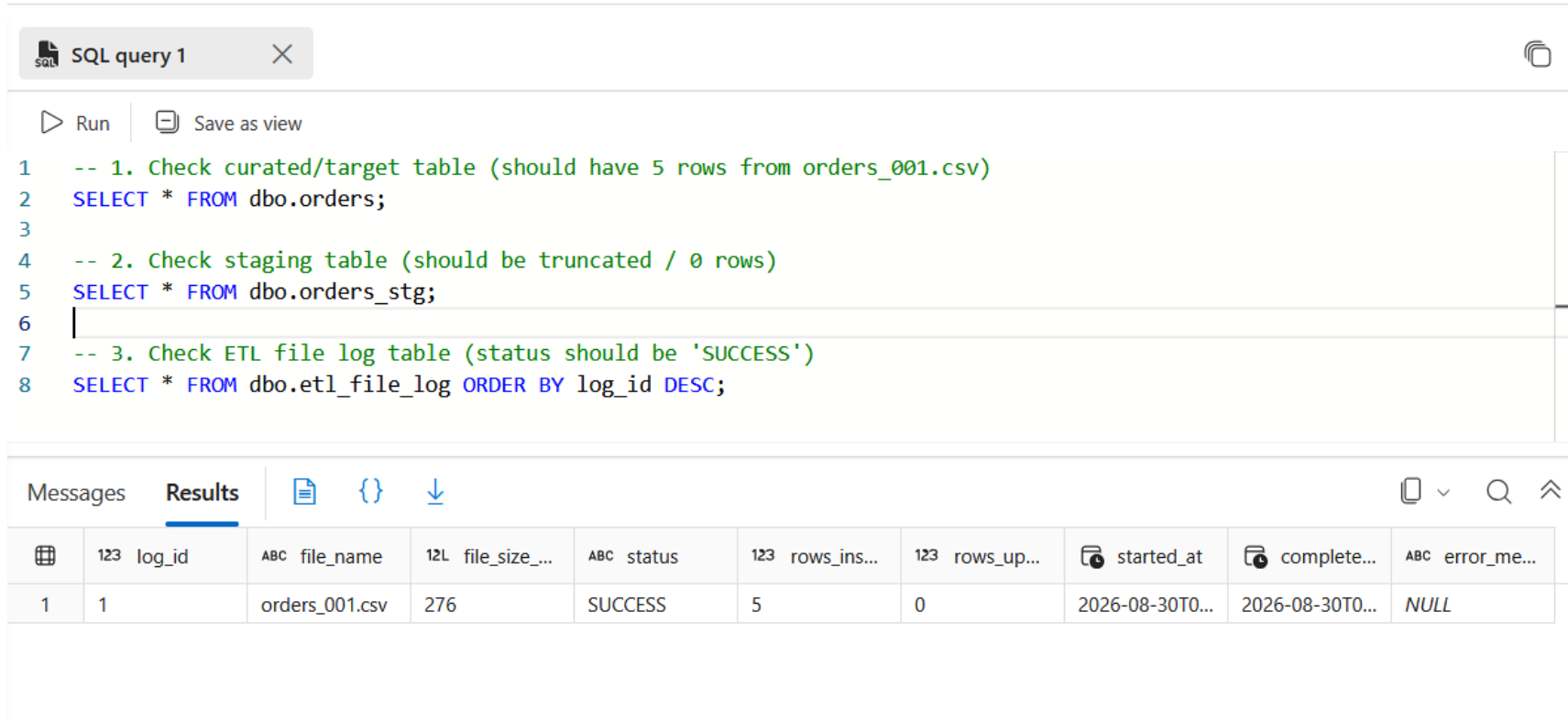
Successful pipeline after fixes

The screenshot displays the Microsoft Fabric Data Factory Author interface. The left sidebar contains navigation links: Home, Author, Monitor, Manage, and Learning Center. The main workspace is divided into three panes. The 'Factory Resources' pane on the left shows a tree view with 'Pipelines' expanded, listing 'PL_INGEST_ORDERS_BATCH'. The 'Activities' pane in the center lists various activity types like 'Move and transform', 'Synapse', 'Azure Data Explorer', etc. The right pane shows the pipeline 'PL_INGEST_ORDERS_BATCH' with a visual flow diagram. The flow starts with a 'data' source, followed by 'Copy_Archive_File', 'Delete_Landing_File', and 'Script_Log_Success' activities, each marked with a green checkmark. Below the flow diagram, the 'Output' tab is active, showing the pipeline status as 'Succeeded' and a table of activity results.

Activity name	Activity status	Activity type	Run start	Duration	Integration
Script_Log_Success	✓ Succeeded	Script	8/30/2026, 11:13:08 AM	12s	AutoResolve
Delete_Landing_File	✓ Succeeded	Delete	8/30/2026, 11:12:45 AM	22s	AutoResolve
Copy_Archive_File	✓ Succeeded	Copy data	8/30/2026, 11:12:27 AM	16s	AutoResolve
SP_Merge_Orders	✓ Succeeded	Stored procedure	8/30/2026, 11:12:08 AM	18s	AutoResolve

The pipeline completed through MERGE, archive copy, landing delete and SUCCESS logging. What this proves: The previously failing configuration was corrected and all eight activities completed.

SQL verification evidence



The screenshot shows a SQL query editor window titled "SQL query 1". The query is as follows:

```
1  -- 1. Check curated/target table (should have 5 rows from orders_001.csv)
2  SELECT * FROM dbo.orders;
3
4  -- 2. Check staging table (should be truncated / 0 rows)
5  SELECT * FROM dbo.orders_stg;
6
7  -- 3. Check ETL file log table (status should be 'SUCCESS')
8  SELECT * FROM dbo.etl_file_log ORDER BY log_id DESC;
```

Below the query editor, the "Results" tab is active, displaying a table with the following data:

	123 log_id	ABC file_name	12L file_size...	ABC status	123 rows_ins...	123 rows_up...	started_at	complete...	error_me...
1	1	orders_001.csv	276	SUCCESS	5	0	2026-08-30T0...	2026-08-30T0...	NULL

Query Editor shows the ETL file log with SUCCESS and 5 inserted rows.
What this proves: Use SQL results as independent verification of ADF execution.

Final successful ADF run

The screenshot displays the Azure Data Factory Author interface for a pipeline named 'PL_INGEST_ORDER...'. The left sidebar shows navigation options: Home, Author, Monitor, Manage, and Learning Center. The 'Factory Resources' pane lists various resources, including Pipelines (1), Datasets (3), Data flows (0), and Power Query (0). The 'Activities' pane shows a workflow with three activities: 'Copy_Archive_File', 'Delete_Landing_File', and 'Script_Log_Success'. The main area displays the 'Output' tab for the pipeline run, showing a 'Succeeded' status and a table of activity details.

Activity name	Activity status	Activity type	Run start	Duration	Integration
Script_Log_Success	Succeeded	Script	8/30/2026, 4:58:43 PM	22s	AutoResolve
Delete_Landing_File	Succeeded	Delete	8/30/2026, 4:58:35 PM	6s	AutoResolve
Copy_Archive_File	Succeeded	Copy data	8/30/2026, 4:58:14 PM	20s	AutoResolve
SP_Merge_Orders	Succeeded	Stored procedure	8/30/2026, 4:57:57 PM	16s	AutoResolve

Final pipeline evidence showing completed activities.

What this proves: This is the end-to-end success state to reproduce in a future run.

Idempotency, archive and missing-file tests

Idempotency

- Re-upload the same orders_001.csv to the same landing path.
- Run the pipeline again.
- Query `SELECT COUNT(*) FROM dbo.orders`.
- Pass: count remains 5, not 10, because MERGE matches on order_id.

Archive

- After success, archive/orders/2026/08/28/orders_001.csv exists.
- The landing copy is deleted only after archive succeeds.

Missing file

- Run again without re-uploading.
- The pipeline should follow Fail_File_Not_Found. This is a controlled negative test.

RBAC controlled-failure test

- Re-upload the test file.
- Temporarily remove Storage Blob Data Contributor from the ADF Managed Identity only.
- Run ADF and capture the 403/authorization failure.
- Restore the role assignment.
- Retry the same pipeline and verify success.

What this test proves

The pipeline is genuinely authorized through Managed Identity + RBAC. Removing authorization produces an observable failure; restoring the correct permission allows safe retry.

11. Errors encountered and fixes - Part 1

'az' is not recognized	azure-identity is a Python SDK, not Azure CLI. Install Microsoft.AzureCLI separately, reopen the terminal, and verify az version.
Azure SQL region/capacity restriction	Choose a region/configuration available to the subscription. Retry SQL server/database creation there.
Script_Log_Start timeout / failure	Check SQL server firewall and the beginner-lab Allow Azure services and resources option. Then verify Microsoft Entra SQL configuration.
SQL Managed Identity login fails	Set a Microsoft Entra admin, connect to the target database, CREATE USER for the exact ADF resource name, then grant required database permissions.

Errors encountered and fixes - Part 2

Get Metadata says file absent	Verify p_container, p_folder and p_file exactly match the ADLS path. For this manual run: landing/orders/2026/08/28/orders_001.csv.
Python upload returns 403	Assign Storage Blob Data Contributor to the signed-in developer identity, not only to ADF.
ADF storage access returns 403	Assign or restore Storage Blob Data Contributor to the ADF system-assigned Managed Identity.
Duplicate rerun concern	The curated table primary/business key is order_id and MERGE updates/matches, so the manual 5-row sample remains 5 rows.
Archive copy succeeds but landing remains	Verify Delete activity dependency, original landing dataset parameters and delete permission.

Two subtle lessons that prevent false debugging

1. Manual staging table may still contain 5 rows

The manual stored procedure shown in the setup does not delete staging rows after MERGE. The Copy Activity truncates staging before the next load. Therefore `dbo.orders_stg` can legitimately contain 5 rows immediately after success. Do not treat that alone as a pipeline failure.

2. `order_date` and `order_ts` are different variants

The manual pipeline uses `order_date`. The repository `generate_orders.py` uses the advanced `order_ts` schema. Do not upload the advanced generator output into the manual mapping unless you intentionally update SQL tables, datasets and mappings.

12. Repository files - what each part is for

python/upload_to_adls.py	Use with the completed manual POC. Uploads a local CSV using DefaultAzureCredential.
python/generate_orders.py	Advanced 30-row generator with deliberate bad rows and order_ts schema.
python/inspect_orders.py	Validates the advanced generated file locally.
sql/	Advanced tables, rejects, file log, watermark and transaction-safe MERGE.
adf/	Sanitized advanced ADF reference JSON and dataset/linked-service templates.
infra/terraform/	Infrastructure-as-code recreation of the Azure foundation.
docs/	Security, monitoring, troubleshooting, validation, interview and cleanup reference.

Manual vs advanced repository variant

Completed manual variant	5-row CSV; order_date; simple ETL log; pre-copy TRUNCATE; MERGE on order_id; archive/delete; SUCCESS logging.
Advanced repository variant	30-row CSV; order_ts; one type-invalid row; one business-invalid row; orders_rejects; processed-file control; watermark; per-file/run staging cleanup; duplicate branch.

Recommended next upgrade

After you can reproduce the manual POC without help, migrate intentionally to the advanced schema and verify the 30 -> 28 curated-row expectation. Treat this as a second lab, not an invisible change to the completed manual flow.

13. Terraform recreation - what it creates

azurerm_resource_group	POC Resource Group
azurerm_storage_account	StorageV2 with hierarchical namespace and TLS 1.2
azurerm_storage_container	landing, archive and quarantine
azurerm_data_factory	ADF with system-assigned identity
azurerm_role_assignment	ADF -> Storage Blob Data Contributor

Terraform recreation - what it creates - Continued

azurerm_key_vault	RBAC-enabled Key Vault
azurerm_mssql_server / database	Azure SQL logical server and small lab database
azurerm_mssql_firewall_rule	Allow Azure services plus optional developer IP
azurerm_log_analytics_work space	Optional diagnostics workspace

Do not collide with the manual environment

Use a second Resource Group and unique globally scoped names, delete the manual lab first, or deliberately import existing resources. A normal terraform apply cannot simply take ownership of already-existing matching resources.

Terraform - prerequisite and variable workflow

Terraform prerequisites

```
terraform -version  
az login  
az account show -o table  
  
cd infra/terraform  
copy terraform.tfvars.example terraform.tfvars
```

- Use unique Storage Account, Data Factory, SQL Server and Key Vault names.
- Keep terraform.tfvars local; do not commit real SQL admin credentials.
- Set client_ip only when you need direct SQL administration from the laptop.
- Set uploader_object_id only if Terraform should grant your developer identity the ADLS data role.
- For a clean practice environment, use a new Resource Group such as rg-azde-poc01-tf-dev.

Terraform - init, validate, plan and apply

Terraform commands

```
terraform init  
terraform fmt -recursive  
terraform validate  
terraform plan  
terraform apply
```

- Read the plan before applying.
- Confirm the intended subscription and Resource Group.
- Type the Terraform confirmation only after reviewing the resource list.
- After apply, inspect outputs and verify the actual resources in Azure Portal.

Terraform - required post-deployment manual steps

- Set a Microsoft Entra administrator on the Azure SQL logical server.
- Connect to the database as that administrator.
- Run the SQL schema and merge scripts that match the ADF mapping you intend to deploy.
- Create the ADF contained database user.
- Author/import ADF linked services, datasets and pipeline using the same schema variant.
- Test connections and run the same end-to-end validation suite.

Networking note

The Terraform lab uses the Azure SQL AllowAzureServices firewall rule for the beginner public-endpoint design. The hardened extension is ADF Managed VNet + Private Endpoints/Private Link.

Terraform - verification and destroy

Verify and destroy

```
terraform output  
az resource list --resource-group <terraform-resource-group> -o table  
  
# Before cleanup  
terraform plan -destroy  
terraform destroy
```

Cleanup rule

If Terraform created the environment and state is intact, use terraform destroy. For an isolated manual lab, deleting the entire Resource Group is the simplest lifecycle cleanup.

Terraform source - versions.tf

```
terraform {  
  required_version = ">= 1.6.0"  
  
  required_providers {  
    azurerm = {  
      source  = "hashicorp/azurerm"  
      version = "~> 4.0"  
    }  
  }  
}  
  
provider "azurerm" {  
  features {}  
  subscription_id = var.subscription_id  
}
```

Terraform source - variables.tf

```
variable "subscription_id" {
  description = "Azure subscription ID. Prefer ARM_SUBSCRIPTION_ID environment variable when possible."
  type        = string
  default     = null
}

variable "location" {
  description = "Azure region for POC resources."
  type        = string
  default     = "centralindia"
}

variable "resource_group_name" {
  type        = string
  default     = "rg-azde-poc01-dev"
}

variable "storage_account_name" {
  description = "Globally unique, lowercase letters/numbers only."
  type        = string
}
```

Terraform source - variables.tf - Continued

```
variable "data_factory_name" {
  description = "Globally unique Data Factory name."
  type        = string
}

variable "sql_server_name" {
  description = "Globally unique Azure SQL logical server name (without .database.windows.net)."
  type        = string
}

variable "sql_database_name" {
  type        = string
  default     = "sqlldb-azde-poc01-dev"
}

variable "sql_admin_login" {
  description = "Bootstrap SQL admin login. Do not commit the real value."
  type        = string
  sensitive   = true
}

variable "sql_admin_password" {
```

Terraform source - variables.tf - Continued

```
    description = "Bootstrap SQL admin password. Do not commit the real value."
    type        = string
    sensitive    = true
}

variable "key_vault_name" {
    description = "Globally unique Key Vault name."
    type        = string
}

variable "client_ip" {
    description = "Optional public IPv4 address of your laptop for SQL admin bootstrap access."
    type        = string
    default     = null
}

variable "uploader_object_id" {
    description = "Optional Microsoft Entra object ID of your developer user. If set, grants Storage Blob Data Contributor role to the user."
    type        = string
    default     = null
}
```

Terraform source - variables.tf - Continued

```
variable "create_log_analytics" {  
    description = "Create a small Log Analytics workspace for optional diagnostics practice."  
    type        = bool  
    default     = false  
}  
  
variable "tags" {  
    type = map(string)  
    default = {  
        project      = "azure-poc"  
        environment   = "dev"  
        owner         = "personal"  
        autoDelete    = "true"  
    }  
}
```


Terraform source - main.tf

```
data "azurerm_client_config" "current" {}

resource "azurerm_resource_group" "this" {
  name      = var.resource_group_name
  location  = var.location
  tags      = var.tags
}

resource "azurerm_storage_account" "this" {
  name                        = var.storage_account_name
  resource_group_name        = azurerm_resource_group.this.name
  location                   = azurerm_resource_group.this.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"
  account_kind                = "StorageV2"
  is_hns_enabled              = true
  min_tls_version             = "TLS1_2"

  allow_nested_items_to_be_public = false
  public_network_access_enabled   = true

  tags = var.tags
}
```

Terraform source - main.tf - Continued

```
}

resource "azurerm_storage_container" "landing" {
  name                       = "landing"
  storage_account_id        = azurerm_storage_account.this.id
  container_access_type     = "private"
}

resource "azurerm_storage_container" "archive" {
  name                       = "archive"
  storage_account_id        = azurerm_storage_account.this.id
  container_access_type     = "private"
}

resource "azurerm_storage_container" "quarantine" {
  name                       = "quarantine"
  storage_account_id        = azurerm_storage_account.this.id
  container_access_type     = "private"
}

resource "azurerm_data_factory" "this" {
  name                       = var.data_factory_name
}
```

Terraform source - main.tf - Continued

```
location          = azurerm_resource_group.this.location
resource_group_name = azurerm_resource_group.this.name

identity {
  type = "SystemAssigned"
}

tags = var.tags
}

resource "azurerm_role_assignment" "adf_storage" {
  scope                = azurerm_storage_account.this.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_data_factory.this.identity[0].principal_id
}

resource "azurerm_role_assignment" "developer_storage" {
  count                = var.uploader_object_id == null ? 0 : 1
  scope                = azurerm_storage_account.this.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = var.uploader_object_id
}
```

Terraform source - main.tf - Continued

```
resource "azurerm_key_vault" "this" {
  name                = var.key_vault_name
  location            = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name
  tenant_id           = data.azurerm_client_config.current.tenant_id
  sku_name            = "standard"
  enable_rbac_authorization = true
  soft_delete_retention_days = 7

  public_network_access_enabled = true

  tags = var.tags
}

# The main POC data path does not need a Key Vault secret because Managed Identity is preferred.
# This role lets the ADF identity read secrets later if you intentionally add a connector that requires one
resource "azurerm_role_assignment" "adf_key_vault" {
  scope                = azurerm_key_vault.this.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = azurerm_data_factory.this.identity[0].principal_id
}
```

Terraform source - main.tf - Continued

```
resource "azurerm_mssql_server" "this" {
  name                        = var.sql_server_name
  resource_group_name        = azurerm_resource_group.this.name
  location                   = azurerm_resource_group.this.location
  version                    = "12.0"
  administrator_login        = var.sql_admin_login
  administrator_login_password = var.sql_admin_password
  minimum_tls_version        = "1.2"
  public_network_access_enabled = true

  tags = var.tags
}

resource "azurerm_mssql_database" "this" {
  name          = var.sql_database_name
  server_id     = azurerm_mssql_server.this.id
  sku_name      = "Basic"
  max_size_gb   = 2

  tags = var.tags
}
```

Terraform source - main.tf - Continued

```
# Beginner-lab networking compromise: lets Azure services such as ADF Azure IR reach Azure SQL.
# Replace this with managed private endpoints / Private Link in the hardened version.
resource "azurerm_mssql_firewall_rule" "allow_azure_services" {
  name          = "AllowAzureServices"
  server_id     = azurerm_mssql_server.this.id
  start_ip_address = "0.0.0.0"
  end_ip_address  = "0.0.0.0"
}

resource "azurerm_mssql_firewall_rule" "developer_ip" {
  count          = var.client_ip == null ? 0 : 1
  name           = "DeveloperLaptop"
  server_id      = azurerm_mssql_server.this.id
  start_ip_address = var.client_ip
  end_ip_address  = var.client_ip
}

resource "azurerm_log_analytics_workspace" "this" {
  count          = var.create_log_analytics ? 1 : 0
  name           = "law-azde-poc01-dev"
  location       = azurerm_resource_group.this.location
}
```

Terraform source - main.tf - Continued

```
resource_group_name = azurerm_resource_group.this.name
sku                 = "PerGB2018"
retention_in_days   = 30

tags = var.tags
}
```

Terraform source - outputs.tf

```
output "resource_group_name" {
  value = azurerm_resource_group.this.name
}

output "storage_account_name" {
  value = azurerm_storage_account.this.name
}

output "storage_dfs_url" {
  value = azurerm_storage_account.this.primary_dfs_endpoint
}

output "data_factory_name" {
  value = azurerm_data_factory.this.name
}

output "data_factory_managed_identity_object_id" {
  value = azurerm_data_factory.this.identity[0].principal_id
}

output "sql_server_fqdn" {
  value = azurerm_mssql_server.this.fully_qualified_domain_name
}
```


Terraform source - outputs.tf - Continued

```
}  
  
output "sql_database_name" {  
    value = azurerm_mssql_database.this.name  
}  
  
output "key_vault_uri" {  
    value = azurerm_key_vault.this.vault_uri  
}
```

Terraform source - terraform.tfvars.example

```
subscription_id = "<YOUR_SUBSCRIPTION_ID>"
location        = "centralindia"
resource_group_name = "rg-azde-poc01-dev"

storage_account_name = "stazdepoc01CHANGE"
data_factory_name     = "adf-azde-poc01-CHANGE"
sql_server_name       = "sql-azde-poc01-CHANGE"
sql_database_name     = "sqlldb-azde-poc01-dev"
key_vault_name        = "kv-azde-poc01-CHANGE"

# NEVER commit real credentials. Copy this file to terraform.tfvars locally.
sql_admin_login      = "pocsqladmin"
sql_admin_password   = "CHANGE_ME_LOCALLY"

# Optional: your current public IPv4 address for SQL client access.
client_ip = null

# Optional: your Microsoft Entra object ID so the local Python uploader has blob data access.
uploader_object_id = null

create_log_analytics = false
```

14. Monitoring checklist

- ADF Studio -> Monitor -> Pipeline runs.
- Capture pipeline name, Run ID, status, start/end time and duration.
- Open Copy_Orders_To_Staging and record Rows read and Rows written.
- For failed runs, capture the first failing activity, error code/message and duration.
- Verify SQL counts and ETL log independently.
- Verify landing/archive paths directly in ADLS.

Optional Log Analytics

Create a small Log Analytics workspace only if budget permits. Route selected ADF diagnostics and remove the lab after evidence capture.

Security and GitHub hygiene

- Prefer Managed Identity when the target supports Microsoft Entra authentication.
- Do not commit .env, terraform.tfvars, *.tfstate, Storage keys, SAS tokens, SQL passwords or application client secrets.
- Assign the narrowest role at the narrowest practical scope.
- The beginner lab uses Storage Blob Data Contributor because ADF must read landing, write archive/quarantine and delete processed landing files.
- Key Vault is useful when a real secret is unavoidable, but the main data path does not require one.

Cleanup - stop POC costs

Azure CLI cleanup

```
az group show -n rg-azde-poc01-dev -o table  
az resource list -g rg-azde-poc01-dev -o table  
  
# Delete only after evidence capture  
az group delete -n rg-azde-poc01-dev --yes
```

- Capture only sanitized architecture, successful run, SQL counts, archive evidence and controlled failures.
- Confirm no unrelated resources are in the Resource Group before deleting.
- After deletion, verify no separately created resource remains outside the POC Resource Group.

15. Interview questions - Part 1

Why land files in ADLS before Azure SQL?

It decouples ingestion from relational processing, preserves raw evidence for audit/replay and lets downstream processing retry without asking the source system to resend.

How is the pipeline idempotent?

The curated table uses order_id as the business key with MERGE update/insert behavior. The advanced repository adds file-level processed control as a second layer.

What is a watermark?

A persisted marker of successful progress, usually a timestamp or source key. The advanced repository updates the watermark only inside a successful transaction.

Managed Identity vs Key Vault secret?

Managed Identity is preferred when the target supports Microsoft Entra authentication because there is no application secret to store or rotate.

Why use staging?

It isolates ingestion from curated logic and makes validation, replay, troubleshooting and transaction-controlled merge easier.

Interview questions - Part 2

When is SHIR required?

When ADF must reach on-premises, local or network-isolated sources that Azure Integration Runtime cannot reach directly.

Why test by removing RBAC?

It proves authorization is real, failures are observable and restoring the correct permission allows safe retry.

How would this scale to thousands of files?

Use metadata-driven control, folder enumeration, ForEach with controlled parallelism, partitioned landing conventions, triggers, retry policies and appropriate compute for heavy transformations.

How would Private Endpoints change the design?

Restrict public endpoints, use ADF Managed Virtual Network and managed private endpoints to ADLS/SQL/Key Vault, while retaining RBAC and SQL database authorization.

Azure RBAC vs SQL permissions?

Azure RBAC controls Azure resource/data-plane access such as ADLS. Azure SQL has a separate database authorization model, so the ADF identity must also exist as a database principal.

POC completion checklist

- ☐ Resource Group created and tagged
- ☐ ADLS Gen2 created with landing/archive/quarantine
- ☐ Azure SQL logical server/database created
- ☐ SQL firewall and Microsoft Entra admin configured
- ☐ ADF created with system-assigned Managed Identity
- ☐ ADF Storage Blob Data Contributor assigned
- ☐ ADF SQL contained user created
- ☐ Linked services test successfully
- ☐ Datasets parameterized

POC completion checklist - Continued

- [] PL_INGEST_ORDERS_BATCH validated and published
- [] 5-row manual file processed successfully
- [] SQL count remains 5 on duplicate rerun
- [] Archive and landing-delete behavior verified
- [] Missing-file negative test captured
- [] RBAC failure/recovery test captured
- [] Python upload works through DefaultAzureCredential
- [] Terraform workflow understood/tested separately
- [] Resource cleanup completed after evidence capture

Beginner replay sequence - one-page revision

Replay sequence

1. Create Resource Group and optional budget
2. Create ADLS Gen2 + landing/archive/quarantine
3. Create Azure SQL server + database
4. Configure firewall + Microsoft Entra admin
5. Create ADF and assign Storage Blob Data Contributor
6. Create SQL tables + usp_merge_orders + ADF database user
7. Create ADF linked services
8. Create parameterized datasets
9. Build PL_INGEST_ORDERS_BATCH
10. Validate + Publish
11. Upload 5-row order_date CSV
12. Run ADF
13. Verify dbo.orders count = 5 and ETL log SUCCESS
14. Verify archive and landing delete
15. Re-upload same file and prove count stays 5
16. Run missing-file and RBAC negative tests
17. Install Azure CLI; az login; run Python uploader
18. Re-run ADF and verify Monitor + SQL
19. Recreate separately with Terraform
20. Capture sanitized evidence and clean up

Final result

The POC demonstrates a secure, parameterized Azure batch ingestion pattern from ADLS Gen2 through Azure Data Factory into Azure SQL, with idempotent business-key MERGE behavior, raw-file archival, execution logging, Managed Identity/RBAC security, Python-based landing and a Terraform recreation path.

Success criteria to remember

A beginner replay is successful when the source lands in ADLS, ADF completes the intended activities, `dbo.orders` contains the expected business rows without duplicates, the ETL log shows SUCCESS, the raw file is archived, negative tests fail predictably, and the environment can be cleaned up safely.